

Package ‘SCP’

July 13, 2026

Type Package

Title Power Evaluation and Study Design for Circadian Biomarker Detection

Version 0.4.57

Date 2026-07-07

Maintainer Thien Quy Pham <quythien14@gmail.com>

URL <https://github.com/quythien/SCP>

BugReports <https://github.com/quythien/SCP/issues>

Description Simulation-based power analysis for circadian transcriptomics studies. Empirical parameter distributions (amplitude, noise, phase, proportion rhythmic) are estimated from a pilot dataset, and power is evaluated for rhythmicity detection and differential-rhythm tests (DR, DP, DM) across a range of study designs. Detection uses a unified cosinor F-test `detect_cosinor(K)`, with $K = 1$ for the single-harmonic case and $K = 2$ for the two-harmonic (ultradian) extension. A bootstrap design-grid optimiser is included for design comparison under pilot uncertainty.

Depends R ($\geq 4.4.0$)

Imports ggplot2,
reshape2,
limma,
limorhyde,
minpack.lm,
nloptr,
parallel,
grid,
grDevices,
stats,
graphics,
utils,
Rcpp

LinkingTo Rcpp,
RcppArmadillo

Suggests shiny,
shinyscreenshot,
DT,
writexl,

enrichR,
 org.Hs.eg.db,
 org.Mm.eg.db,
 AnnotationDbi,
 FMM,
 MetaCycle,
 rain,
 circacompare,
 patchwork,
 cowplot,
 gtable,
 readxl,
 tidyr,
 AWFisher,
 DODR,
 BiocManager,
 knitr,
 rmarkdown,
 testthat ($\geq 3.0.0$)

License MIT + file LICENSE

Encoding UTF-8

LazyData false

biocViews Software, GeneExpression, TimeCourse, DifferentialExpression, Transcriptomics

RoxygenNote 7.3.2

VignetteBuilder knitr

Contents

.buildBioFromBoot	5
.collapseTail	5
.computeMarginalPower	6
.load_pilot_data	7
.prepDiffStratified	7
.resampleSubjectIndices	8
.selectTimePoints	8
.vcat	9
add_seBars	9
adjust.to.2pi	10
alpha.adjust	10
amp.cut	11
bootstrapParams	11
calculate_CI.M	12
calculate_CI_A.phase.Taylor	12
choose.abs.min	13
CircadianAnalysisOptions	13
CircadianBioOptions	14
CircadianBootstrapOptions	17
CircadianDesignOptions	17
CircaPower	18
circaPowerApproxN80	19

circular_concentration	20
circular_difference	20
circular_mean	21
compareDesignApproaches	21
compare_methods	22
CP_OneGroup	22
DCP_Analyze	23
DCP_DiffPar	24
DCP_DiffR2	25
DCP_Rhythmicity	26
DCP_sim_data	27
detect_CircaCompare	28
detect_cosinor	29
detect_DCP	30
detect_DCP_diff	30
detect_DODR	31
detect_FMM	31
detect_JTK	33
detect_LimoRhyde	34
detect_MH	34
detect_RAIN	35
estCircadianParam	36
estCircadianParam2H	37
estCircadianParamFMM	39
estCircadianParamTwoGroup	41
estimate_r_from_data	42
false_discovery_cost	43
fishers.p	43
fitCosinorAll	44
fitSinCurve	44
format_for_DCP	46
forwardStop	46
generatePilotData	47
generate_power_report	47
get_phase	48
get_r_stratum	48
launchShiny	49
LR_deltaR2	50
LR_diff	51
makeAdaptiveRStrata	51
npower	52
one_cosinor_OLS	53
plot.SCPDiffResult	54
plot.SCPRecommendResult	54
plot.SCPSingleResult	55
plotBootstrapDesignGrid	55
plotBvsMPower	56
plotDensityResults	57
plotDesignComparison	57
plotDiffPower	58
plotDRPowerStratified	59
plotFMMDifferential	60

plotFMMViolation	61
plotFourierDeviation	62
plotGroundTruthComparison	62
plotSingleCohortPower	63
plot_power_curves	65
plot_stratified_power	65
plot_two_group_heatmap	66
power_assessment	66
prepCircadianData	67
print.CircadianAnalysisOptions	68
print.CircadianBioOptions	69
print.CircadianBootstrapOptions	69
print.CircadianDesignOptions	70
print.estimate_r_from_data	70
print.npower	71
print.SCPDiffResult	71
print.SCPRecommendResult	72
print.SCPSingleResult	72
printDensitySummary	73
printMethodGuidance	73
rayleigh_test_circular	74
recommendDesign	74
replotDRPower	75
runBootstrapDesignGrid	76
runDifferentialPower	78
runFourierDeviationPower	79
runGroundTruthComparison	80
runPhaseShiftAnalysis	81
runPowerAnalysis	81
runSCPValidation	82
runSimsDiff	82
runSimsSingleCohort	84
runSingleCohortGrid	86
runSingleCohortPower	87
runTwoStagePower	88
run_DCP_pipeline	89
sampleTimesFromDist	90
scp_load_pilot	90
scp_pilots	91
scp_pilot_database	92
scp_pilot_search	93
scp_pilot_tod	93
SeqModelSel	94
simCircadianDiff	95
simCircadianDiffFMM	97
simCircadianFMM	98
simCircadianSingleCohort	99
simCircadianSingleCohort2H	100
StopRule	101
stratified_power	102
summarizeDCPPairs	102
SummarizeDR	103

summaryBootstrapDesignGrid	104
summaryRunPower	104
targeted_power	105
testAnyDifferential	105
toTOJR	106
updateBioOptions	107
updateSimOptions	107
Index	108

`.buildBioFromBoot` *Build CircadianBioOptions from a Bootstrap Parameter Draw*

Description

Constructs a `CircadianBioOptions` object from a single bootstrap resample of pilot-gene cosinor fits. Differential parameters (`prop_DR`, `prop_DP`, `phase_diff`, etc.) are inherited from `bio_diff.opts`. Amplitude and sigma are kept jointly sampled to preserve the empirical A-sigma correlation.

Usage

```
.buildBioFromBoot (boot_df, bio_diff.opts)
```

Arguments

`boot_df` Data frame. One bootstrap resample of `fitCosinorAll()` output (columns: M, A, phi, sigma, `is_rhythmic`).

`bio_diff.opts` List or `CircadianBioOptions`. Source for differential-design parameters (`prop_DR`, `prop_DP`, `phase_diff`, etc.).

Value

A `CircadianBioOptions` object ready for `simCircadianSingleCohort()`.

`.collapseTail` *Collapse High-r Tail Strata into a Single Bin*

Description

Merges all r-strata whose left boundary is at or above `threshold` into a single "`r >= threshold`" bin. This prevents sparse high-r strata from cluttering stratified differential power plots.

Usage

```
.collapseTail(
  r_breaks,
  strata_labels,
  gene_counts,
  mean_TD,
  se_TD,
  mean_pow,
  se_pow,
  threshold = 5
)
```

Arguments

<code>r_breaks</code>	Numeric vector. Stratum breakpoints (length = <code>n_strata</code> + 1).
<code>strata_labels</code>	Character vector. Labels for each stratum.
<code>gene_counts</code>	Integer vector. Gene counts per stratum per simulation.
<code>mean_TD</code>	Numeric vector. Mean true discoveries per stratum.
<code>se_TD</code>	Numeric vector. SE of true discoveries per stratum.
<code>mean_pow</code>	Numeric vector. Mean power per stratum.
<code>se_pow</code>	Numeric vector. SE of power per stratum.
<code>threshold</code>	Numeric. <code>r</code> value above which strata are collapsed (default 5).

Value

Named list with collapsed vectors: `strata_labels`, `gene_counts`, `mean_TD`, `se_TD`, `mean_pow`, `se_pow`, and scalar `n_strata`.

```
.computeMarginalPower
```

Internal helper: compute marginal power from `runSimsDiff()` output

Description

Internal helper: compute marginal power from `runSimsDiff()` output

Usage

```
.computeMarginalPower(sim_out, test_type, fdr_threshold, nsims)
```

Arguments

<code>sim_out</code>	Output from <code>runSimsDiff()</code> with <code>sample_sizes</code> of length 1
<code>test_type</code>	"DR", "DP", or "DM"
<code>fdr_threshold</code>	Numeric threshold
<code>nsims</code>	Number of simulations (3rd dim of <code>fdr</code> array)

Value

Numeric vector of length nsims (power per simulation)

<code>.load_pilot_data</code>	<i>Load built-in pilot dataset by name</i>
-------------------------------	--

Description

Load built-in pilot dataset by name

Usage

```
.load_pilot_data(name)
```

Arguments

name	Dataset name (e.g., "ball_ba47_younger")
------	--

Value

List with lBaselineExpr, IOD, amplitude, phase, prop_rhythmic

<code>.prepDiffStratified</code>	<i>Prepare Differential Power Data for Stratified Plotting</i>
----------------------------------	--

Description

Converts a raw `runSimsDiff()` result into stratified power and true-discovery arrays indexed by sample size, r-stratum, FDR threshold, and simulation replicate.

Usage

```
.prepDiffStratified(res, ep, fdr_thresholds, r_breaks)
```

Arguments

res	List. Output of <code>runSimsDiff()</code> (contains fdr_DR, fdr_DP, fdr_DM, diff_type, effectsizes).
ep	Character. Endpoint: one of "DR", "DP", "DM".
fdr_thresholds	Numeric vector. FDR thresholds to evaluate power at.
r_breaks	Numeric vector. Breakpoints defining r-strata (e.g. <code>c(0, 0.5, 1, 2, Inf)</code>).

Value

Named list with arrays `power_arr`, `TD_arr`, `marginal_sim`, and matrix `count_mat`.

```
.resampleSubjectIndices
```

Reproducible subject-resample indices for the subject bootstrap

Description

Draws `nboot` sets of `n_pilot` subject indices with replacement (Efron nonparametric bootstrap over subjects). The RNG state is saved and restored so the draws are reproducible from `seed` without disturbing the caller's random stream.

Usage

```
.resampleSubjectIndices(n_pilot, nboot, seed = 42)
```

Arguments

<code>n_pilot</code>	Number of pilot subjects (columns of the pilot matrix).
<code>nboot</code>	Number of bootstrap draws.
<code>seed</code>	Random seed.

Value

List of length `nboot`; each element is an integer index vector of length `n_pilot`.

```
.selectTimePoints
```

Select B Evenly-Spaced Time Points from a Design Vector

Description

Select B Evenly-Spaced Time Points from a Design Vector

Usage

```
.selectTimePoints(design_vector, B)
```

Arguments

<code>design_vector</code>	Numeric vector. Full set of candidate ZT time points.
<code>B</code>	Integer. Number of distinct time bins to select.

Value

Numeric vector of length `min(B, length(design_vector))` with evenly-spaced time points drawn from `design_vector`.

<code>.vcat</code>	<i>Print a pass/fail validation line</i>
--------------------	--

Description

Print a pass/fail validation line

Usage

```
.vcat(label, pass, detail = NULL)
```

Arguments

<code>label</code>	Character label for the check being reported.
<code>pass</code>	Logical; TRUE prints "PASS", FALSE prints "FAIL".
<code>detail</code>	Optional character detail line printed indented below.

Value

Invisibly NULL; called for its printing side effect.

<code>add_se_bars</code>	=====
	<i>Plotting Functions with Standard Error Bars</i>
	=====

Description

SE plotting functions for DR/DP stratified power and phase shift analyses. $SE = sd(power) / \sqrt{nsims}$ computed across simulation replicates.

Usage

```
add_se_bars(x, y, se, col, bar_width = 0.3)
```

Arguments

<code>x</code>	Numeric vector. x-coordinates of the bar centres.
<code>y</code>	Numeric vector. y-coordinates (point estimates).
<code>se</code>	Numeric vector. Standard errors (bar half-width).
<code>col</code>	Colour passed to <code>arrows()</code> .
<code>bar_width</code>	Numeric. Relative width of the bar caps (default 0.3).

Details

Functions: `add_se_bars()` - Add +/-1 SE error bars to a plot `plotWithSE()` - 6-panel DR/DP power figure with SE bars `plotPhaseShiftWithSE()` - 6-panel phase shift figure with SE bars Add +/-1 SE Error Bars to a Base-R Plot

Value

Invisibly returns NULL. Called for its side-effect of drawing arrows on the active plot device.

adjust.to.2pi	<i>Wrap Angle to [0, 2pi)</i>
---------------	-------------------------------

Description

Wrap Angle to [0, 2pi)

Usage

adjust.to.2pi(x)

Arguments

x Numeric. Angle(s) in radians.

Value

Numeric. Angle(s) mapped to [0, 2pi).

alpha.adjust	<i>Alpha Adjustment for Multiple Testing</i>
--------------	--

Description

Alpha Adjustment for Multiple Testing

Usage

alpha.adjust(alpha, k, method = "bonferroni")

Arguments

alpha	Original alpha
k	Number of tests
method	"bonferroni" or "sidak"

Value

Adjusted alpha

amp.cut

Amplitude Cutoff Adjustment

Description

Amplitude Cutoff Adjustment

Usage

```
amp.cut(amp, a.TOJR)
```

Arguments

amp	Vector indicating if amplitudes exceed cutoff
a.TOJR	Current TOJR category

Value

Adjusted TOJR category

bootstrapParams

Bootstrap parameter sets from pilot fit

Description

Resamples G rows (genes) with replacement from param_df to generate nboot independent parameter draws.

Usage

```
bootstrapParams(param_df, nboot, seed = 42)
```

Arguments

param_df	Data frame from fitCosinorAll() (G rows)
nboot	Number of bootstrap draws
seed	Random seed

Value

List of length nboot; each element is a data frame like param_df

calculate_CI.M	<i>Calculate CI for Mesor</i>
----------------	-------------------------------

Description

Calculate CI for Mesor

Usage

```
calculate_CI.M(XX.inv, A.t, r.full = 6, n, alpha = 0.05, sigma.hat)
```

Arguments

XX.inv	Inverse of the design matrix cross-product (t(X) times X).
A.t	Contrast row vector selecting the mesor.
r.full	Residual degrees-of-freedom offset (default 6).
n	Number of observations.
alpha	Significance level (default 0.05).
sigma.hat	Estimated residual standard deviation.

Value

CI half-width (radius) for the mesor estimate.

calculate_CI_A.phase.Taylor	<i>Calculate CI for Amplitude and Phase (Taylor method)</i>
-----------------------------	---

Description

Calculate CI for Amplitude and Phase (Taylor method)

Usage

```
calculate_CI_A.phase.Taylor(XX.inv, A.t, phase.hat, A.hat, sigma.hat)
```

Arguments

XX.inv	Inverse of the design matrix cross-product (t(X) times X).
A.t	Contrast row vector selecting the cosinor coefficients.
phase.hat	Estimated phase (radians).
A.hat	Estimated amplitude.
sigma.hat	Estimated residual standard deviation.

Value

List with `se.A.hat` and `se.phase.hat`, the delta-method standard errors for amplitude and phase.

choose.abs.min	<i>Choose the minimum absolute value from each row</i>
----------------	--

Description

Choose the minimum absolute value from each row

Usage

```
choose.abs.min(mat = matrix(stats::rnorm(21), ncol = 3))
```

Arguments

mat Numeric matrix; one candidate value per column, one case per row.

Value

Numeric vector, length `nrow(mat)` : the smallest-magnitude entry from each row (sign preserved).

CircadianAnalysisOptions	<i>Create Analysis & Reporting Options</i>
--------------------------	--

Description

Create Analysis & Reporting Options

Usage

```
CircadianAnalysisOptions(
  alpha = 0.05,
  p.adjust.method = "BH",
  parallel.ncores = 1,
  amp.cutoff = 0,
  target_effect = 0.1,
  fdr_thresholds = c(0.01, 0.05, 0.1, 0.2),
  reference_n = 60,
  r_strata = c(0, 0.25, 0.5, 0.75, 1, 1.25, 1.5, 1.75, 2, 2.5, 3, 3.5, 4, 4.5, 5),
  strata_labels = NULL,
  phase_shifts = c(0, 0.5, 1, 2, 4, 6, 8, 10, 12),
  DCmethod = "DCP"
)
```

Arguments

<code>alpha</code>	Significance level for DCP pipeline
<code>p.adjust.method</code>	Multiple testing correction method
<code>parallel.ncores</code>	Number of cores for parallel DCP
<code>amp.cutoff</code>	Amplitude cutoff for DCP_Rhythmicity
<code>target_effect</code>	Minimum effect size to count as "interesting"
<code>fdr_thresholds</code>	FDR thresholds for power curves
<code>reference_n</code>	Reference sample size for Panel B / phase shift plots
<code>r_strata</code>	Breakpoints for A/sigma stratification
<code>strata_labels</code>	Labels for strata (auto-generated if NULL)
<code>phase_shifts</code>	Phase shift magnitudes to sweep (for sensitivity analysis)
<code>DCmethod</code>	Differential-rhythmicity detector to report against, one of "DCP" (default) or "CircaCompare".

Value

Object of class "CircadianAnalysisOptions"

CircadianBioOptions

Create Biology + Differential Options

Description

Create Biology + Differential Options

Usage

```
CircadianBioOptions(
  ngenes = 5000,
  prop_rhythmic = NULL,
  period = 24,
  lBaselineExpr = NULL,
  lBaselineExpr2 = NULL,
  lod = NULL,
  lod2 = NULL,
  amplitude = NULL,
  amplitude2 = NULL,
  sigma_rhythmic = NULL,
  paired_sigma = FALSE,
  paired_omega = FALSE,
  paired_alpha = FALSE,
  omega_rhythmic = NULL,
```

```

alpha_rhythmic = NULL,
omega_dist = NULL,
alpha_dist = NULL,
cts = NULL,
cts2 = NULL,
phase = "uniform",
prop_DR = 0.15,
prop_DP = 0.1,
prop_DM = 0,
phase_diff = c(-6, 6),
amp_diff = c(0.5, 2),
mesor_diff = c(0.5, 2),
dp_shift_mode = c("fixed", "uniform"),
dr_amp_scale = 1,
dr_sigma_scale = 1,
alpha_pilot = NULL,
sim.seed = 12345
)

```

Arguments

<code>ngenes</code>	Number of genes.
<code>prop_rhythmic</code>	Proportion of rhythmic genes. If <code>NULL</code> and a pilot dataset name is supplied to <code>lBaselineExpr</code> , the pilot's estimated proportion is used.
<code>period</code>	Circadian period in hours.
<code>lBaselineExpr</code>	Log baseline expression. Can be a numeric scalar (constant for all genes), a numeric vector (resampled to <code>ngenes</code>), a function <code>f(n)</code> returning <code>n</code> values, or a character string naming a built-in pilot dataset stored in <code>data/</code> . No default, must be supplied.
<code>lBaselineExpr2</code>	As <code>lBaselineExpr</code> but for group 2 (differential analyses only). <code>NULL</code> uses the same distribution as group 1.
<code>lOD</code>	Log over-dispersion (noise). Same forms accepted as <code>lBaselineExpr</code> . No default, must be supplied.
<code>lOD2</code>	As <code>lOD</code> but for group 2. <code>NULL</code> shares group 1 values.
<code>amplitude</code>	Amplitude distribution for rhythmic genes. Same forms accepted as <code>lBaselineExpr</code> . No default, must be supplied.
<code>amplitude2</code>	As <code>amplitude</code> but for group 2.
<code>sigma_rhythmic</code>	Optional numeric vector of per-gene noise values for rhythmic genes (same length as <code>amplitude</code>). When provided, <code>amplitude</code> and <code>sigma</code> are drawn jointly to preserve pilot A-sigma correlation.
<code>paired_sigma</code>	Logical. If <code>TRUE</code> and <code>sigma_rhythmic</code> is supplied, expand <code>amplitude</code> and <code>sigma_rhythmic</code> jointly by the same resampled gene index so their pilot-estimated pairwise correlation is preserved; if <code>FALSE</code> (default), <code>amplitude</code> and <code>sigma</code> are expanded independently.
<code>paired_omega</code>	Logical. If <code>TRUE</code> and <code>omega_rhythmic</code> is supplied (same length as <code>amplitude</code>), expand <code>omega</code> using the same shared gene-index used for <code>amplitude/sigma</code> so all three are paired per gene.

<code>paired_alpha</code>	Logical. Same as <code>paired_omega</code> for <code>alpha_rhythmic</code> .
<code>omega_rhythmic</code>	Optional numeric vector of per-gene FMM omega values from a pilot FMM fit (same length as <code>amplitude</code>). Used when <code>paired_omega = TRUE</code> ; ignored when <code>omega_dist</code> is also set.
<code>alpha_rhythmic</code>	Optional numeric vector of per-gene FMM alpha values in radians from a pilot FMM fit (same length as <code>amplitude</code>).
<code>omega_dist</code>	Optional list specifying a distribution from which omega is drawn at simulation time, e.g. <code>list(family="beta", a=1, b=5)</code> or <code>list(family="fixed", value=0.7)</code> . Overrides <code>omega_rhythmic</code> when both are provided.
<code>alpha_dist</code>	Optional list specifying a distribution for alpha, <code>list(family="normal", mean=0, sd_hours=2)</code> . Internally <code>sd_hours</code> is converted to radians via <code>sd_rad = sd_hours * 2*pi/24</code> . When <code>alpha_rhythmic</code> is also provided and lengths match, the perturbation is added to each gene's empirical alpha (<code>alpha_g = alpha_rhythmic[g] + N(0, sd_rad^2)</code>). When <code>alpha_rhythmic</code> is missing, <code>alpha_g = N(0, sd_rad^2)</code> .
<code>cts</code>	Numeric vector of sample collection times for passive design.
<code>cts2</code>	As <code>cts</code> for group 2.
<code>phase</code>	Phase distribution for rhythmic genes. "uniform" (default) or the same forms as <code>lBaselineExpr</code> .
<code>prop_DR</code>	Proportion with differential rhythmicity.
<code>prop_DP</code>	Proportion with differential phase.
<code>prop_DM</code>	Proportion with differential mesor (mean shift, both groups rhythmic).
<code>phase_diff</code>	Range of phase shift for DP genes <code>c(min, max)</code> .
<code>amp_diff</code>	Range of amplitude ratio (unused; retained for interface compatibility).
<code>mesor_diff</code>	Range of mesor shift for DM genes <code>c(min, max)</code> (additive, log-scale units).
<code>dp_shift_mode</code>	"fixed" (use <code>phase_diff[2]</code>) or "uniform" (sample uniformly within <code>phase_diff</code> range).
<code>dr_amp_scale</code>	Scale factor for amplitude (A) to adjust DR strength.
<code>dr_sigma_scale</code>	Scale factor for sigma to adjust DR strength.
<code>alpha_pilot</code>	Rhythmicity pre-screen threshold used to define the pilot's rhythmic gene set (the value at which <code>prop_rhythmic</code> and the effect-size distribution were calibrated). Stored on the object for provenance and reporting; default <code>NULL</code> .
<code>sim.seed</code>	Random seed.

Value

Object of class "CircadianBioOptions".

CircadianBootstrapOptions

Create Bootstrap Design Grid Options

Description

Create Bootstrap Design Grid Options

Usage

```
CircadianBootstrapOptions (
  design_vector,
  B_values = c(4, 6, 8, 12, 24),
  m_values = NULL,
  N_values = NULL,
  nboot = 200,
  nsims_inner = 20,
  design = c("active", "passive"),
  seed = 42
)
```

Arguments

design_vector	Ordered time points to sweep over (length $\geq \max(B_values)$)
B_values	Number of distinct time points to test
m_values	Replicates per time point (if NULL, derived as $\text{round}(N/B)$)
N_values	Total per-group sample sizes to test (if NULL, derived as $B*m$)
nboot	Number of bootstrap parameter draws (outer uncertainty loop)
nsims_inner	Simulations per bootstrap draw
design	"active" or "passive"
seed	Random seed

Value

Object of class "CircadianBootstrapOptions"

CircadianDesignOptions

Create Study Design Options

Description

Create Study Design Options

Usage

```
CircadianDesignOptions(
  sample_sizes = c(10, 20, 40, 60, 80, 100),
  nsims = 100,
  design = c("active", "passive"),
  cts = NULL,
  B_values = NULL,
  test_types = c("DR", "DP", "DM"),
  omega = 1,
  beta = pi
)
```

Arguments

<code>sample_sizes</code>	Vector of sample sizes per group
<code>nsims</code>	Number of simulation replicates
<code>design</code>	"active" or "passive"
<code>cts</code>	Time-of-day distribution for passive design (numeric vector)
<code>B_values</code>	Optional vector of time-point counts (B) to sweep, in addition to <code>sample_sizes</code> ; NULL runs a single B implied by <code>cts</code> .
<code>test_types</code>	Character vector of tests to run ("DR", "DP", "DM")
<code>omega</code>	FMM waveform shape parameter in (0, 1]. <code>omega = 1</code> (default) is the pure cosinor path; <code>omega < 1</code> simulates a non-sinusoidal waveform via <code>simCircadianFMM()</code> . <code>omega = 0</code> is not allowed.
<code>beta</code>	FMM orientation parameter (peak-location offset, default <code>pi</code>); only used when <code>omega < 1</code> , ignored for the cosinor path.

Value

Object of class "CircadianDesignOptions"

CircaPower

Utility Functions for Circadian Power Analysis

Description

Fast closed-form power calculation based on F-test. Useful for quick estimates before running full simulations. Based on CircaPower's analytical approach.

Usage

```
CircaPower(n = NULL, power = NULL, r = NULL, alpha = 0.05, design_factor = 0.5)
```

Arguments

<code>n</code>	Sample size (NULL to solve for n)
<code>power</code>	Target power (NULL to solve for power)
<code>r</code>	Effect size (A/σ)
<code>alpha</code>	Significance level
<code>design_factor</code>	Sampling design factor (0.5 for even spacing)

Details

Quick analytical power for single-group rhythmicity

Value

List with n, power, r, alpha

Examples

```
# How many samples for 80% power?
CircaPower(n = NULL, power = 0.8, r = 1.5)

# What power with n=24?
CircaPower(n = 24, power = NULL, r = 1.2)
```

```
circaPowerApproxN80
```

Estimate n for 80 percent power using CircaPower at median pilot r

Description

Estimate n for 80 percent power using CircaPower at median pilot r

Usage

```
circaPowerApproxN80(
  bio.opts,
  alpha = 0.05,
  target_power = 0.8,
  n_search = seq(5, 500, by = 5)
)
```

Arguments

bio.opts	CircadianBioOptions with amplitude and sigma_rhythmic.
alpha	Significance level (default 0.05).
target_power	Target power (default 0.80).
n_search	Integer vector of candidate sample sizes.

Value

Single integer – smallest n achieving target_power, or NA if not found.

`circular_concentration`*Circular concentration (1 - circular variance)*

Description

Circular concentration (1 - circular variance)

Usage

```
circular_concentration(angles, period = 24)
```

Arguments

<code>angles</code>	Angles (in hours)
<code>period</code>	Period (default 24)

Value

Concentration (0 = uniform, 1 = all same)

`circular_difference`*Signed circular difference between two phases*

Description

Signed circular difference between two phases

Usage

```
circular_difference(phi1, phi2, period = 24)
```

Arguments

<code>phi1, phi2</code>	Phases (same units as period, e.g. hours).
<code>period</code>	Period used to wrap the difference (default 24).

Value

`phi1 - phi2` wrapped to `[-period/2, period/2]`.

circular_mean	<i>Circular mean calculation</i>
---------------	----------------------------------

Description

Circular mean calculation

Usage

```
circular_mean(angles, period = 24)
```

Arguments

angles	Angles (in hours)
period	Period (default 24)

Value

Mean angle (in hours)

compareDesignApproaches	<i>Compare two-stage vs bootstrap power estimates</i>
-------------------------	---

Description

Merges per-N results from both approaches into a comparison data frame. For bootstrap results, averages over B values to get marginal power per N.

Usage

```
compareDesignApproaches (
  two_stage_result,
  bootstrap_result,
  test_type = "DR",
  target_power = 0.8
)
```

Arguments

two_stage_result	Output from runTwoStagePower()
bootstrap_result	Output from runBootstrapDesignGrid()
test_type	Test type to extract from bootstrap result
target_power	Target power level for n80 calculation

Value

List with comparison data frame and n_80 summaries

compare_methods	<i>Compare Multiple Methods</i>
-----------------	---------------------------------

Description

Compare power, FDR, and FDC across methods

Usage

```
compare_methods(power_results_list)
```

Arguments

power_results_list	List of results from multiple methods
--------------------	---------------------------------------

Value

Comparison summary table

CP_OneGroup	<i>Single Group Rhythmicity Analysis (Internal)</i>
-------------	---

Description

Single Group Rhythmicity Analysis (Internal)

Usage

```
CP_OneGroup(x1, period = 24, alpha = 0.05, CI = FALSE, p.adjust.method = "BH")
```

Arguments

x1	Group 1 data (list with data, time, gname components)
period	Period (default 24)
alpha	Significance threshold
CI	Calculate confidence intervals?
p.adjust.method	P-value adjustment method

Value

List with rhythmicity results

Description

Performs differential tests (DR, DP) with classification

Usage

```
DCP_Analyze(
  expr1,
  expr2,
  times1,
  times2,
  alpha = 0.05,
  gene_names = NULL,
  parallel.ncores = 1
)
```

Arguments

expr1	Expression matrix for group 1 (genes x samples)
expr2	Expression matrix for group 2 (genes x samples)
times1	Time points for group 1
times2	Time points for group 2
alpha	Significance threshold (default 0.05)
gene_names	Gene names (optional)
parallel.ncores	Number of cores for parallel processing

Value

List with:

DR	Differential rhythmicity results
DP	Differential phase results
classification	Gene classifications (DR/DP/NS)

Examples

```
x = DCP_sim_data(ngene=1000, nsample=30, A1=c(1, 3), A2=c(1, 3),
  phase1=c(0, pi/4), phase2=c(pi/4, pi/2),
  M1=c(4, 6), M2=c(4, 6), sigma1=1, sigma2=1)
dcp_result = DCP_Analyze(x[[1]]$data, x[[2]]$data,
  x[[1]]$time, x[[2]]$time)
```

DCP_DiffPar	<i>Differential Parameter Test (DP and DA)</i>
-------------	--

Description

Tests for differences in amplitude, phase, and/or mesor between two groups

Usage

```
DCP_DiffPar (
  x,
  Par = c("A"),
  TOJR = NULL,
  alpha = 0.05,
  p.adjust.method = "BH",
  parallel.ncores = 1
)
```

Arguments

x	Output from DCP_Rhythmicity(x1, x2)
Par	One of "A", "phase", "M", "A&phase", or "A&phase&M"
TOJR	TOJR output. If NULL, rhythm.joint object in x will be used
alpha	P-value cutoff
p.adjust.method	P-value adjustment method
parallel.ncores	Number of cores for parallel processing

Value

Data frame with differential parameter test results

Examples

```
x = DCP_sim_data(ngene=1000, nsample=30, A1=c(1, 3), A2=c(1, 3),
  phase1=c(0, pi/4), phase2=c(pi/4, pi/2),
  M1=c(4, 6), M2=c(4, 6), sigma1=1, sigma2=1)
rhythm.res = DCP_Rhythmicity(x1 = x[[1]], x2 = x[[2]])
rhythm.diffPar = DCP_DiffPar(rhythm.res, Par = "A&phase")
```


DCP_DiffR2

*Differential Rhythmicity Test (DR)***Description**

Tests for differential rhythmicity between two groups using R^2 difference

Usage

```
DCP_DiffR2(
  x,
  method = "LR",
  TOJR = NULL,
  alpha = 0.05,
  nSampling = 1000,
  Sampling.save = NULL,
  Sampling.file.label = "gII_vs_gI",
  p.adjust.method = "BH",
  parallel.ncores = 1
)
```

Arguments

x	Output from DCP_Rhythmicity(x1, x2)
method	One of "LR" (recommended) or "LR_sigma2"
TOJR	TOJR output. If NULL, rhythm.joint object in x will be used
alpha	P-value cutoff
nSampling	Number of samplings for permutation/bootstrap
Sampling.save	Directory to save sampling results
Sampling.file.label	File label for saved results
p.adjust.method	P-value adjustment method
parallel.ncores	Number of cores for parallel processing

Value

Data frame with differential R^2 test results

Examples

```
x = DCP_sim_data(ngene=1000, nsample=30, A1=c(1, 3), A2=c(1, 3),
  phase1=c(0, pi/4), phase2=c(pi/4, pi/2),
  M1=c(4, 6), M2=c(4, 6), sigma1=1, sigma2=1)
rhythm.res = DCP_Rhythmicity(x1 = x[[1]], x2 = x[[2]])
rhythm.diffR2 = DCP_DiffR2(rhythm.res)
```

Description

This function either takes single-group data and performs only rhythmicity analysis, or takes two-group data and also categorize the genes into types of joint rhythmicity (TOJR).

Usage

```
DCP_Rhythmicity(
  x1,
  x2 = NULL,
  method = "Sidak_FS",
  period = 24,
  amp.cutoff = 0,
  alpha = 0.05,
  alpha.FDR = 0.05,
  CI = FALSE,
  p.adjust.method = "BH",
  parallel.ncores = 1
)
```

Arguments

<code>x1</code>	Group 1 data (list with data, time, gname components)
<code>x2</code>	Group 2 data (optional, same format as <code>x1</code>)
<code>method</code>	Algorithm for joint rhythmicity categorization. Should be one of "Sidak_FS" (default, recommended), "Sidak_BS", "VDA", "AWFisher"
<code>period</code>	Oscillation cycle length (default 24 for circadian)
<code>amp.cutoff</code>	Only genes with amplitude > <code>amp.cutoff</code> are considered rhythmic
<code>alpha</code>	Threshold for rhythmicity p-value
<code>alpha.FDR</code>	Threshold for FDR-adjusted rhythmicity p-value
<code>CI</code>	Calculate confidence intervals?
<code>p.adjust.method</code>	P-value adjustment method
<code>parallel.ncores</code>	Number of cores for parallel processing

Details

The methods "Sidak_FS" and "Sidak_BS" implement selective sequential model selection with Sidak adjusted p-value. "FS" represents forward stop, and "BS" basic stop. The method "Sidak_FS" has better type I error control compared to Venn diagram analysis (VDA) and adaptively weighted Fisher's method (AWFisher).

Value

List with:

`x1, x2` Input data with added rhythmicity analysis
`gname_overlap` Overlapping gene names
`rhythm.joint` Data frame with joint rhythmicity categories

References

Fithian, W., Taylor, J., Tibshirani, R., & Tibshirani, R. (2015). Selective sequential model selection. arXiv preprint arXiv:1512.02565.

Examples

```
x = DCP_sim_data(ngene=1000, nsample=30, A1=c(1, 3), A2=c(1, 3),
  phase1=c(0, pi/4), phase2=c(pi/4, pi/2),
  M1=c(4, 6), M2=c(4, 6), sigma1=1, sigma2=1)
rhythm.res = DCP_Rhythmicity(x1 = x[[1]], x2 = x[[2]])
```

DCP_sim_data

*Simulate Data for Rhythmicity Comparison***Description**

Simple two-group study simulation for illustration

Usage

```
DCP_sim_data(
  ngene = 1000,
  nsample = 30,
  A1 = c(1, 3),
  A2 = c(1, 3),
  phase1 = c(0, 2 * pi),
  phase2 = c(0, 2 * pi),
  M1 = c(4, 6),
  M2 = c(4, 6),
  sigma1 = 1,
  sigma2 = 1
)
```

Arguments

`ngene` Number of genes
`nsample` Number of samples per group
`A1` Amplitude range for group 1 (c(min, max))
`A2` Amplitude range for group 2
`phase1` Phase range for group 1

phase2	Phase range for group 2
M1	MESOR range for group 1
M2	MESOR range for group 2
sigma1	Noise level for group 1
sigma2	Noise level for group 2

Value

List of two elements, each is a list with data, time, gname components

Examples

```
x = DCP_sim_data(ngene=1000, nsample=30, A1=c(1, 3), A2=c(1, 3),
phase1=c(0, pi/4), phase2=c(pi/4, pi/2),
M1=c(4, 6), M2=c(4, 6), sigma1=1, sigma2=1)
```

```
detect_CircaCompare
```

CircaCompare Differential Analysis Wrapper

Description

Per-gene NLS fitting with group-interaction parameters. Model: $y \sim (k + k1*group) + (\alpha + \alpha1*group) * \cos(\tau*t - (\phi + \phi1*group))$ Provides separate Wald t-tests for amplitude diff ($\alpha1 \rightarrow DA$), phase diff ($\phi1 \rightarrow DP$), and mesor diff ($k1 \rightarrow DM$).

Usage

```
detect_CircaCompare(
  expr1,
  times1,
  expr2,
  times2,
  gene_names = NULL,
  period = 24,
  alpha_threshold = 0.05
)
```

Arguments

expr1	Expression matrix for group 1 (genes x samples)
times1	Time points for group 1
expr2	Expression matrix for group 2 (genes x samples)
times2	Time points for group 2
gene_names	Gene name vector (optional)
period	Circadian period (default 24)
alpha_threshold	Significance level for per-group rhythmicity (default 0.05)

Details

CircaCompare calls `stop()` when either group is arrhythmic, so every gene call is wrapped in `tryCatch`. NLS failures also yield $p=1$ (conservative). CircaCompare has no differential rhythmicity (DR) test – only per-group rhythmicity p -values. `pval_DR` is set to NA for all genes.

Value

List with `pval_DR` (NA – no equivalent), `pval_DP`, `pval_DM` Each is a numeric vector of length `ngenes`; default 1 for failed/untested.

<code>detect_cosinor</code>	<i>Unified cosinor rhythmicity detection (single- or multi-harmonic)</i>
-----------------------------	--

Description

Canonical entry point for rhythmicity detection by K -harmonic cosinor regression. $K = 1$ is the single-harmonic cosinor F-test, identical to [detect_DCP](#); $K = 2$ adds the 12-hour second harmonic and tests the joint harmonic block. By default the unshrunk exact nested F-test is used (matching the power derivations in the paper). Returns raw p -values; the caller applies FDR control.

Usage

```
detect_cosinor(expr, times, K = 1L, period = 24, ebayes = FALSE, mc.cores = 1L)
```

Arguments

<code>expr</code>	Gene x sample expression matrix
<code>times</code>	Numeric time vector (length = <code>ncol(expr)</code>)
<code>K</code>	Harmonic order (default 1)
<code>period</code>	Period in hours (default 24)
<code>ebayes</code>	For $K \geq 2$, moderate residual variance with empirical-Bayes shrinkage (default FALSE, the unshrunk exact F-test)
<code>mc.cores</code>	Cores for the $K \geq 2$ fit (default 1)

Value

Numeric vector of raw p -values, length `nrow(expr)`

See Also

[detect_DCP](#)

detect_DCP	<i>DCP single-cohort rhythmicity detection</i>
------------	--

Description

DCP single-cohort rhythmicity detection

Usage

```
detect_DCP(expr, times, period = 24)
```

Arguments

expr	Gene x sample expression matrix
times	Numeric time vector (length = ncol(expr))
period	Period (default 24)

Value

Numeric vector of p-values, length nrow(expr)

detect_DCP_diff	<i>DCP two-group differential detection</i>
-----------------	---

Description

Wraps run_DCP_pipeline() (TOJR -> DiffR2 -> DiffPar). DM is not tested by DCP; pval_DM is returned as NA.

Usage

```
detect_DCP_diff(expr1, times1, expr2, times2, period = 24)
```

Arguments

expr1, expr2	Gene x sample matrices for groups 1 and 2
times1, times2	Numeric time vectors
period	Period (default 24)

Value

list(pval_DR, pval_DP, pval_DM=NA) each numeric[G]

detect_DODR	<i>DODR two-group differential oscillation detection</i>
-------------	--

Description

Wraps DODR::dodr(). DR only.

Usage

```
detect_DODR(expr1, times1, expr2, times2, period = 24)
```

Arguments

expr1, expr2	Gene x sample matrices
times1, times2	Numeric time vectors
period	Period (default 24)

Value

list(pval_DR, pval_DP=NA, pval_DM=NA)

detect_FMM	<i>FMM-Harmonic F-test for Rhythmicity Detection</i>
------------	--

Description

Tests for rhythmicity in time-course data using the K-harmonic Fourier decomposition of the FMM (Frequency Modulated Mobius) signal model. Equivalent to a 2K-parameter joint cosinor regression test under $H_0 : a_1 = b_1 = \dots = a_K = b_K = 0$. With `ebayes = TRUE` (default), variance estimates are moderated by `limma::eBayes` following the LimoRhyde framework (Singer & Hughey 2019), extended here to user-specified K .

Usage

```
detect_FMM(
  expr,
  times,
  period = 24,
  K = 2L,
  ebayes = TRUE,
  adjust.method = "BH",
  fdr.threshold = 0.05,
  return.coefficients = FALSE,
  mc.cores = 1L
)
```

Arguments

<code>expr</code>	Numeric matrix (genes x samples) of expression values.
<code>times</code>	Numeric vector of sample collection times in hours (length = <code>ncol(expr)</code>).
<code>period</code>	Period in hours (default 24).
<code>K</code>	Number of harmonics to include (default 2). $K=1$ reduces to standard cosinor (DCP). $K=2$ is the recommended default for non-cosinor signals; $K=3$ may improve power for very sharp peaks ($\omega < 0.15$). Identifiability requires the design to have at least $2 \cdot K + 1$ distinct sampling phases per period ($B \geq 3$ for $K=1$, $B \geq 5$ for $K=2$, $B \geq 7$ for $K=3$). <code>detect_FMM</code> can be called stand-alone on any gene-by-sample expression matrix; the function emits a warning when the supplied design violates the identifiability bound.
<code>ebayes</code>	Logical. If <code>TRUE</code> (default), variance is moderated across genes via <code>limma::eBayes</code> . If <code>FALSE</code> , the unshrunk exact F-test is used (useful for null-calibration verification). The function automatically falls back to the unshrunk path when <code>nrow(expr) < 50</code> .
<code>adjust.method</code>	Multiple-testing adjustment for the BH-style discovery flag (default "BH"). Set to <code>NULL</code> or "none" to skip.
<code>fdr.threshold</code>	FDR cutoff for the discovery column (default 0.05). Ignored if <code>adjust.method</code> is <code>NULL</code> /"none".
<code>return.coefficients</code>	If <code>TRUE</code> , the returned data frame includes per-gene harmonic coefficients <code>a_1</code> , <code>b_1</code> , ..., <code>a_K</code> , <code>b_K</code> .
<code>mc.cores</code>	Parallel cores. Currently unused; the linear-regression path is fully vectorised across genes. Argument retained for API compatibility with other detectors.

Details

The test fits the linear model

$$y_t = M + \sum_{k=1}^K [a_k \cos(k 2\pi t/T) + b_k \sin(k 2\pi t/T)] + \varepsilon$$

and tests the joint null $H_0 : a_1 = b_1 = \dots = a_K = b_K = 0$. With `ebayes = TRUE`, fitting and inference proceed via `limma::lmFit` on a basis assembled from per-harmonic calls to `limorhyde::limorhyde(times, period=T/k)` for $k = 1, \dots, K$, followed by `limma::eBayes` variance shrinkage and `limma::topTable` extraction of the joint harmonic F-test. With `ebayes = FALSE`, the same model is fit by a vectorised QR decomposition and tested by the exact nested F-statistic with null distribution $F(2K, n - 2K - 1)$ under Gaussianity.

This is motivated by the FMM model's closed-form Fourier expansion: the cosine of a Mobius-transformed angle has harmonic amplitudes that decay geometrically with $r = (1 - \omega)/(1 + \omega)$. Truncating at K and releasing the decay constraint trades a small bias (the truncation error r^{2K}) for exact identifiability under H_0 , avoiding the Davies-supremum calibration problem of a direct nonlinear FMM fit.

Value

Data frame with one row per gene:

`F.stat` Numerator-over-denominator F statistic.

df1, df2 Numerator (2K) and denominator (n-2K-1, or the moderated denominator df when ebayes = TRUE) degrees of freedom.

p.value F-test p-value (moderated when ebayes = TRUE).

p.adjust Adjusted p-value (if adjust.method non-null).

R2 Coefficient of determination of the K-harmonic fit.

discovery Logical: p.adjust < fdr.threshold.

References

Rueda, C., Larriba, Y., Peddada, S. D. (2019). Frequency Modulated Mobius Model for the Estimation of Rhythmic Signals. Sci Rep 9, 18138.

Singer, J. M., Hughey, J. J. (2019). LimoRhyde: a flexible approach for differential analysis of rhythmic transcriptome data. J Biol Rhythms 34(1), 5-18.

Hughes, M. E. et al. (2017). Guidelines for genome-scale analysis of biological rhythms. J Biol Rhythms 32(5), 380-393.

Examples

```
set.seed(1)
n <- 48
tt <- seq(0, 24*(1-1/n), length.out = n)
rhy <- 2 * cos(2*pi*tt/24) + 0.6 * cos(4*pi*tt/24) # 1st + 2nd harmonics
expr <- rbind(
  rhythmic = rhy + rnorm(n, 0, 0.5),
  arrhythmic = rnorm(n, 0, 0.5)
)
detect_FMM(expr, tt, K = 2)
```

detect_JTK	<i>JTK_CYCLE Rhythmicity Detection</i>
------------	--

Description

Non-parametric rhythmicity test based on Kendall's tau-b. Uses MetaCycle::meta2d with cycMethod = "JTK".

Usage

```
detect_JTK(expr, times, gene_names = NULL, period = 24)
```

Arguments

expr	Expression matrix (genes x samples)
times	Time points (numeric vector, hours)
gene_names	Gene name vector (optional)
period	Period to test (default 24)

Details

JTK_CYCLE is designed for evenly-spaced time series. For passive designs with uneven sampling, MetaCycle handles the conversion internally, but power may be reduced compared to evenly-spaced data.

Value

Numeric vector of adjusted p-values per gene (length ngenes). Returns 1 for genes where the test fails.

detect_LimoRhyde	<i>LimoRhyde two-group differential rhythmicity</i>
------------------	---

Description

limma interaction model: group x cosinor (cos + sin terms). Tests H0: no group x rhythm interaction. DR only.

Usage

```
detect_LimoRhyde(expr1, times1, expr2, times2, period = 24)
```

Arguments

expr1, expr2	Gene x sample matrices
times1, times2	Numeric time vectors
period	Period (default 24)

Value

```
list(pval_DR, pval_DP=NA, pval_DM=NA)
```

detect_MH	<i>Multi-harmonic rhythmicity detection (adaptive $K = \text{floor}((B-1)/2)$)</i>
-----------	---

Description

Fits K harmonics where B = number of unique time points. Tests all harmonic terms jointly via an F-test against intercept-only null.

Usage

```
detect_MH(expr, times, period = 24)
```

Arguments

expr	Gene x sample expression matrix
times	Numeric time vector (length = ncol(expr))
period	Period (default 24)

Value

Numeric vector of p-values, length nrow(expr)

Note

Superseded by `detect_FMM`, which supports explicit K, the LimoRhyde-style `limma::lmFit + limma::eBayes` backend with cross-gene variance shrinkage, and a richer return value (per-gene F, df1, df2, R2, p.adjust, discovery). Retained for backwards compatibility; new code should call `detect_FMM(expr, times, K = <K>, ebayes = TRUE/FALSE)` instead.

detect_RAIN	<i>RAIN Rhythmicity Detection</i>
-------------	-----------------------------------

Description

Non-parametric rhythmicity test allowing asymmetric waveforms. Uses the `rain::rain` function from Bioconductor.

Usage

```
detect_RAIN(expr, times, gene_names = NULL, period = 24)
```

Arguments

<code>expr</code>	Expression matrix (genes x samples)
<code>times</code>	Time points (numeric vector, hours)
<code>gene_names</code>	Gene name vector (optional)
<code>period</code>	Period to test (default 24)

Details

RAIN is designed for evenly-spaced time series. For uneven sampling, data is sorted by time and the median interval is used as `deltat`. Power may be reduced compared to evenly-spaced designs.

Value

Numeric vector of p-values per gene (length ngenes). Returns 1 for genes where the test fails.

estCircadianParam *Estimate Circadian Parameters from Pilot Data (Single-Cohort)*

Description

Fits a cosinor model per gene, ranks rhythmic genes by p-value, and returns empirical parameter distributions packaged as a `CircadianBioOptions` object suitable for downstream simulation and power analysis.

Usage

```
estCircadianParam(
  data,
  times,
  period = 24,
  min_rhythm_pval = 0.01,
  top_k = 300L,
  prop_DR = 0.15,
  prop_DP = 0.1,
  prop_DM = 0,
  mesor_diff = c(0.5, 2),
  phase_diff = c(-6, 6),
  amp_diff = c(0.5, 2),
  dp_shift_mode = c("fixed", "uniform"),
  dr_amp_scale = 1,
  dr_sigma_scale = 1,
  paired_sigma = FALSE,
  sim.seed = 12345,
  verbose = TRUE
)
```

Arguments

<code>data</code>	Numeric matrix of pilot expression (genes x samples). A <code>data.frame</code> is coerced to matrix.
<code>times</code>	Numeric vector of sample collection times in hours, length <code>ncol(data)</code> . The function does NOT validate units; pass hours (0 to 24 scale).
<code>period</code>	Period in the same units as <code>times</code> (default 24 hours).
<code>min_rhythm_pval</code>	Uncorrected cosinor p-value threshold for the rhythmic-candidate set (default 0.01). Used to compute <code>prop_rhythmic</code> and to select the top-K ($K = \min(\text{top_k}, n_{\text{rhythmic}})$) genes whose amplitudes, phases, and noise drive simulation truth.
<code>top_k</code>	Number of highest-signal genes (ranked by cosinor p-value) that form the effect-size distribution (amplitude, phase, noise). Default 300. If <code>top_k</code> exceeds the number of rhythmic genes, it is capped to that number with a warning. <code>Inf</code> or <code>"all"</code> uses every rhythmic candidate.
<code>prop_DR, prop_DP, prop_DM</code>	Proportions of differentially-rhythmic, differentially-phased, and differentially-mesor genes (defaults 0.15, 0.10, 0.00). Capped automatically at <code>prop_rhythmic</code> .

```
mesor_diff, phase_diff, amp_diff
    Numeric length-2 ranges for the differential effect-size sampling.
dp_shift_mode
    One of "fixed" or "uniform" (default "fixed").
dr_amp_scale, dr_sigma_scale
    Scaling factors for DR-class amplitude and noise distributions (default 1).
paired_sigma
    Logical, default FALSE. If TRUE, joint (A, sigma) sampling preserves pilot
    correlation in downstream simulations.
sim.seed
    Integer seed for downstream simulation (default 12345).
verbose
    Logical, default TRUE. Print summary at end.
```

Value

A `CircadianBioOptions` S3 object usable directly with `runSimsSingleCohort()` or `runDifferentialPower()`.

See Also

[estCircadianParamFMM](#) for FMM-aware pre-screen, [estCircadianParamTwoGroup](#) for two-group setup, [prepCircadianData](#) for normalisation upstream.

Examples

```
## Not run:
set.seed(1)
n_genes <- 200; n_samples <- 30
times <- rep(seq(0, 22, by = 2), length.out = n_samples)
expr <- matrix(rnorm(n_genes * n_samples), nrow = n_genes)
for (g in 1:20) expr[g, ] <- expr[g, ] + 2 * cos(2*pi*times/24)
bio <- estCircadianParam(expr, times, period = 24, verbose = FALSE)

## End(Not run)
```

```
estCircadianParam2H
```

Estimate CircadianBioOptions Under the Two-Harmonic Cosinor Model

Description

Two-harmonic analog of [estCircadianParam](#). Fits the regression

$$y = \mu + a_1 \cos(\omega_0 t) + b_1 \sin(\omega_0 t) + a_2 \cos(2\omega_0 t) + b_2 \sin(2\omega_0 t) + \epsilon$$

to each gene by OLS, tests the joint rhythmicity null $H_0 : a_1 = b_1 = a_2 = b_2 = 0$ on 4 d.f., and returns a `CircadianBioOptions` object whose 5-way joint empirical distribution $F(\mu, A_1, \phi_1, A_2, \phi_2, \sigma)$ is preserved by gene-paired index draws downstream.

The returned object carries the standard 1-harmonic fields (`amplitude`, `phase`, `sigma_rhythmic`) holding the 1st-harmonic $A_{g,1}$, $\phi_{g,1}$, and σ_g respectively, and three extra fields attached to the list after construction:

- `amplitude2`: 2nd-harmonic amplitudes $A_{g,2}$
- `phase2`: 2nd-harmonic acrophases $\phi_{g,2}$ in hours $[0, period/2)$
- `paired_2h`: TRUE flag (downstream simulators read this to switch on the 2-harmonic generative model).

These extras live alongside `$diagnostics`; the constructor is not modified.

Usage

```
estCircadianParam2H(
  data,
  times,
  period = 24,
  min_rhythm_pval = 0.01,
  top_k = 500,
  prop_DR = 0.15,
  prop_DP = 0.1,
  prop_DM = 0,
  mesor_diff = c(0.5, 2),
  phase_diff = c(-6, 6),
  amp_diff = c(0.5, 2),
  dp_shift_mode = c("fixed", "uniform"),
  dr_amp_scale = 1,
  dr_sigma_scale = 1,
  paired_sigma = FALSE,
  sim.seed = 12345,
  verbose = TRUE
)
```

Arguments

<code>data</code>	Numeric matrix of pilot expression (genes x samples).
<code>times</code>	Numeric vector of sample collection times in hours, length <code>ncol(data)</code> .
<code>period</code>	Period in the same units as <code>times</code> (default 24 hours).
<code>min_rhythm_pval</code>	P-value threshold for the joint 4-d.f. F-test (default 0.01). Genes with $p <$ this define the candidate set.
<code>top_k</code>	Cap on number of genes contributing to the joint $(A_1, \phi_1, A_2, \phi_2, \sigma)$ distribution (default 500), ranked by F-test p-value.
<code>prop_DR, prop_DP, prop_DM</code>	Differential proportions (defaults 0.15, 0.10, 0.00). Forwarded unchanged to <code>CircadianBioOptions</code> ; note the K=2 extension itself is single-cohort only.
<code>mesor_diff, phase_diff, amp_diff</code>	Differential effect-size ranges.
<code>dp_shift_mode</code>	"fixed" or "uniform" (default "fixed").
<code>dr_amp_scale, dr_sigma_scale</code>	DR scale factors (default 1).
<code>paired_sigma</code>	Logical (default FALSE). Forwarded to <code>CircadianBioOptions</code> .
<code>sim.seed</code>	Integer seed (default 12345).
<code>verbose</code>	Logical (default TRUE). Print diagnostic summary.

Value

A `CircadianBioOptions` object with extra fields `$amplitude2`, `$phase2`, `$paired_2h`, and a `$diagnostics` list summarising the 2H fit (median A_2/A_1 ratio, fraction of genes with significant 2nd-harmonic block, etc.).

Methodological notes

1. Pilot summary is a 5-way joint empirical distribution. When the simulator draws a rhythmic gene, the same index j is used across $(A_1, \phi_1, A_2, \phi_2, \sigma)$, preserving the cross-parameter dependence observed in the pilot.
2. No 2nd-harmonic significance threshold is applied: ALL top-K genes passing the joint 4-d.f. F-test contribute, even those whose A_2 is essentially noise. The empirical F_{A_2} will have a noise spike near zero, which is fine.
3. K=2 extension is single-cohort only – the differential simulator (`simCircadianDiff`) is not extended.

See Also

`estCircadianParam` for the 1-harmonic case, `simCircadianSingleCohort2H` for the matching simulator.

`estCircadianParamFMM`

Estimate CircadianBioOptions with FMM Per-Gene Parameters

Description

Like `estCircadianParam`, but additionally fits the FMM (Frequency Modulated Mobius) waveform per top-K rhythmic gene to obtain empirical $\hat{\omega}_g$ (shape) and $\hat{\alpha}_g$ (peak location, radians). Returns a `CircadianBioOptions` populated with paired `omega_rhythmic` and `alpha_rhythmic` vectors (paired with `amplitude` and `sigma_rhythmic` by gene index). A `$diagnostics` list carries empirical fit summaries: Beta(1, beta) MoM estimate, `sigma_alpha` (standard deviation of alpha in hours), median R^2 of the FMM fits, and the raw per-gene vectors for diagnostic plotting.

Used by Fig 5 (B-vs-m grid under empirical FMM truth) and the diagnostic supplementary figure. Fig 4 (sensitivity sweep) does NOT need this – it uses synthetic `omega_dist/alpha_dist` draws.

Usage

```
estCircadianParamFMM(
  data,
  times,
  period = 24,
  min_rhythm_pval = 0.01,
  top_k = 300L,
  screen_K = 2L,
  paired_sigma = TRUE,
  paired_omega = TRUE,
  paired_alpha = TRUE,
  prop_DR = 0,
```

```

prop_DP = 0,
prop_DM = 0,
mc.cores = 1L,
verbose = TRUE,
...
)

```

Arguments

<code>data</code>	Gene expression matrix (genes x samples).
<code>times</code>	Sample collection times (length = ncol(data)).
<code>period</code>	Period in hours (default 24).
<code>min_rhythm_pval</code>	<code>detect_FMM</code> pre-screen p-value threshold (default 0.01). Genes with <code>detect_FMM</code> $p <$ this value are classified as rhythmic candidates that contribute to the empirical (A, sigma, omega, alpha) distributions and to <code>prop_rhythmic</code> . Matches the downstream <code>detect_FMM</code> detector (so simulation truth and detection use the same model).
<code>top_k</code>	Cap on number of FMM-fitted genes (default 300, matches <code>estCircadianParam</code> 's top-K logic).
<code>screen_K</code>	Harmonic order for the <code>detect_FMM</code> pre-screen (default 2, i.e. the K=2 harmonic LRT). Must match the downstream detector's harmonic order to keep truth-label generation and detection internally consistent.
<code>paired_sigma</code>	Logical (default TRUE). Forwarded to <code>CircadianBioOptions</code> .
<code>paired_omega</code>	Logical (default TRUE). Forwarded.
<code>paired_alpha</code>	Logical (default TRUE). Forwarded.
<code>prop_DR</code>	Target proportion of differential-rhythmicity genes, forwarded to <code>CircadianBioOptions</code> (default 0).
<code>prop_DP</code>	Target proportion of differential-phase genes, forwarded to <code>CircadianBioOptions</code> (default 0).
<code>prop_DM</code>	Target proportion of differential-MESOR genes, forwarded to <code>CircadianBioOptions</code> (default 0).
<code>mc.cores</code>	Parallel cores for FMM fitting (default 1).
<code>verbose</code>	Print progress (default TRUE).
<code>...</code>	Additional args passed to <code>CircadianBioOptions()</code> .

Value

`CircadianBioOptions` object with: - `amplitude`, `sigma_rhythmic`, `omega_rhythmic`, `alpha_rhythmic` vectors of equal length (= number of successfully fitted top-K genes). **`omega_rhythmic`** is the FMM shape parameter in $[0, 1]$; **`alpha_rhythmic`** is the FMM peak location in radians $[0, 2\pi]$. - \$diagnostics list: `beta_hat`, `sigma_alpha_hat` (hours), `R2_median`, `omega_emp`, `alpha_emp`, `n_fitted`, `screen_method` = "detect_FMM".

FMM scale convention

This function fits `FMM::fitFMM()` on *radians* timepoints $t \in [0, 2\pi]$, because `FMM::fitFMM()` does NOT internally rescale `timePoints`. Empirically, on a known signal with true $\omega = 0.3$: fitting in radians recovers $\hat{\omega} = 0.30$; fitting on $[0, 1]$ gives $\hat{\omega} = 0.046$ (off by $\sim 7\times$ and meaningless as

an FMM shape parameter). Radians is also the convention used by the simulator (`simCircadianFMM: cts_rad = cts * 2*pi/24` and `FMM::generateFMM(..., to = 2*pi)`), so the (`omega`, `alpha`) returned here are directly usable as simulation truth.

Note: the pre-screen below uses the K=2 harmonic LRT (`detect_FMM`), which has the same null hypothesis as the downstream detector. This keeps the truth-label generator and the detector consistent. (`omega`, `alpha`) from the FMM fit are propagated only as simulation truth, not detection.

```
estCircadianParamTwoGroup
```

Estimate a two-group circadian pilot for differential power

Description

Fits cosinor models to two groups of pilot data and assembles the two-group pilot summary used by the differential power simulator, encoding differential rhythmicity (DR), differential phase (DP), and differential MESOR (DM) structure.

Usage

```
estCircadianParamTwoGroup(
  data_1,
  data_2,
  times_1,
  times_2,
  period = 24,
  min_rhythm_pval = 0.01,
  top_k = 300L,
  phase_shift_threshold = 2,
  prop_DM = NULL,
  mesor_diff = NULL,
  dp_shift_mode = c("uniform", "fixed"),
  paired_sigma = FALSE,
  sim.seed = 12345,
  verbose = TRUE
)
```

Arguments

<code>data_1, data_2</code>	Gene-by-sample expression matrices for the two groups.
<code>times_1, times_2</code>	Numeric collection times for each group.
<code>period</code>	Period in hours (default 24).
<code>min_rhythm_pval</code>	Marginal p-value screen defining the rhythmic pilot set.
<code>top_k</code>	Number of highest-signal genes (per group, ranked by cosinor p-value) forming each group's effect-size distribution (default 300). Capped (with a warning) to a group's rhythmic-gene count if larger; <code>Inf</code> or <code>"all"</code> uses every rhythmic candidate.

phase_shift_threshold	Minimum phase difference (hours) for the DP class.
prop_DM	Optional target proportion of differential-MESOR genes.
mesor_diff	Optional MESOR difference for the DM component.
dp_shift_mode	Differential-phase shift model, "uniform" or "fixed".
paired_sigma	Preserve the paired (amplitude, noise) relationship across groups.
sim.seed	Random seed.
verbose	Print progress.

Value

A two-group CircadianBioOptions pilot summary.

estimate_r_from_data	<i>Estimate Signal-to-Noise Ratio (r) from DCP Results</i>
----------------------	--

Description

Estimate the median signal-to-noise ratio $r = A/\sigma$ from the top circadian genes in a DCP (Differential Circadian Pipeline) analysis. Uses the relationship $r = 1/\sqrt{1/R^2 - 1}$ where R^2 is rhythm strength.

Usage

```
estimate_r_from_data(dcp_results, top_n = 100, rank_by = "q.R2", use_group = 1)
```

Arguments

dcp_results	List from DCP analysis containing DR and DP components
top_n	Number of top circadian genes to use (default: 100)
rank_by	Column to rank genes by (default: "q.R2")
use_group	Which group's R^2 to use (1 or 2; default 1).

Value

List with median r, mean r, and individual r values

Examples

```
## Not run:
# dcp_results is the output from runDiffCircadian()
r_estimate <- estimate_r_from_data(dcp_results, top_n = 100)
# Use median r for power simulation
target_r <- r_estimate$median_r

## End(Not run)
```

`false_discovery_cost`*False Discovery Cost (FDC)*

Description

Calculate the cost of each true discovery, measured as the number of false discoveries per true discovery.

Usage

```
false_discovery_cost (TP, FP)
```

Arguments

TP	True positives
----	----------------

FP	False positives
----	-----------------

Value

FDC (FP / TP)

`fishers.p`*Fisher's Combined P-value*

Description

Fisher's Combined P-value

Usage

```
fishers.p(ps)
```

Arguments

ps	Vector of p-values
----	--------------------

Value

Combined p-value

fitCosinorAll

Bootstrap Design Grid for Circadian Power Analysis

Description

PowerSim performs pilot-informed power analysis for cross-sectional circadian studies. Gene-level circadian parameter distributions are estimated from pilot data, and uncertainty in those distributions is propagated through an outer bootstrap. Conditional on a chosen parameter distribution, independent subjects are simulated under a population-average cosinor model for candidate designs.

Usage

```
fitCosinorAll(data, times, period = 24, min_rhythm_pval = 0.01)
```

Arguments

data	Genes x samples matrix
times	Sample time points (length = ncol(data))
period	Circadian period in hours (default 24)
min_rhythm_pval	P-value threshold for classifying rhythmic genes

Details

The bootstrap quantifies uncertainty in the pilot-implied parameter distribution (mesor, amplitude, phase, noise). It does NOT create subject-level variability within a single simulation; that is captured by sigma_g (residual SD from the cosinor fit), which represents pooled between-subject spread conditional on TOD. sigma_g does NOT model subject-specific random effects in M, A, or phi, the model is a population-average cosinor, which is a defensible first-order baseline for cross-sectional post-mortem studies.

Key notation: B = number of distinct time bins (subset of design_vector; must be ≥ 3) m = subjects per time bin $\rightarrow N = B * m$ per-group sample size (exact, no rounding) nboot = outer bootstrap draws for pilot-estimation uncertainty Fit cosinor model to all genes in pilot data

Value

Data frame: gene, M, A, phi, sigma, pvalue, is_rhythmic

fitSinCurve

Differential Circadian Analysis for PowerSim

Description

This file provides differential circadian rhythm analysis methods Based on the DiffCircadian package implementation

Usage

```
fitSinCurve(
  tt,
  yy,
  period = 24,
  parStart = list(amp = 3, phase = 0, offset = 0)
)
```

Arguments

tt	Time vector
yy	Expression vector
period	Period of the sine curve (default 24)
parStart	Initial value for optimization (default list(amp=3,phase=0,offset=0))

Details

Main Functions: DCP_Rhythmicity() - Single/two-group rhythmicity analysis with TOJR DCP_DiffR2()
 - Differential rhythmicity test (DR) DCP_DiffPar() - Differential parameter tests (DP, DA) DCP_Analyze()
 - Complete differential analysis workflow

Author: Based on DiffCircadian package Date: 2025-02-12 Fit Sinusoidal Curve

Fit a sine curve where tt is time, and yy is expression value.

Value

A list with components:

amp	Amplitude
phase	Phase (0 to period)
offset	Basal level
peak	Peak time
A	A coefficient (sin component)
B	B coefficient (cos component)
tss	Total sum of squares
rss	Residual sum of squares
R2	R-squared

Examples

```
set.seed(32608)
n <- 50
tt <- runif(n, 0, 24)
Amp <- 2
Phase <- 6
Offset <- 3
yy <- Amp * sin(2*pi/24 * (tt + Phase)) + Offset + rnorm(n, 0, 1)
fitSinCurve(tt, yy)
```

format_for_DCP	<i>Bridge: Convert simulation output to DCP format</i>
----------------	--

Description

Bridge: Convert simulation output to DCP format

Usage

```
format_for_DCP(expr_matrix, times, gene_names = NULL)
```

Arguments

expr_matrix	Gene x sample matrix
times	Time vector
gene_names	Gene name vector

Value

DCP-formatted list

forwardStop	<i>Forward Stopping Rule</i>
-------------	------------------------------

Description

Forward Stopping Rule

Usage

```
forwardStop(pv, alpha = 0.1)
```

Arguments

pv	P-values
alpha	Threshold

Value

List with stop and p

generatePilotData *Generate synthetic pilot data from known true parameters*

Description

Simulates a single-group cross-sectional dataset (genes x n_pilot) with subject times sampled from pilot_times. Used as a stand-in for real pilot data when the ground truth is known.

Usage

```
generatePilotData(true_bio.opts, n_pilot, pilot_times, seed = 42)
```

Arguments

true_bio.opts	CircadianBioOptions with known true parameters
n_pilot	Number of pilot subjects
pilot_times	TOD distribution to sample subject times from
seed	Random seed

Value

List: data (genes x n_pilot matrix), times (length n_pilot)

generate_power_report
 Generate Power Report

Description

Build a printable power analysis report covering overall and stratified metrics.

Usage

```
generate_power_report(power_results, output_file = NULL)
```

Arguments

power_results	Results from power analysis
output_file	Output file path (NULL = return as text)

Value

Report text or NULL if written to file

get_phase

Get Phase from Beta Coefficients

Description

Get Phase from Beta Coefficients

Usage

```
get_phase(b1.x, b2.x)
```

Arguments

b1.x Cosine-term coefficient.
b2.x Sine-term coefficient.

Value

List with `phase` (radians) and `tan` (the ratio used).

get_r_stratum

Get Recommended r Stratum for Power Simulation

Description

Given estimated `r`, return the appropriate `r` stratum label for use in power simulation stratification.

Usage

```
get_r_stratum(
  r_estimate,
  r_strata = c(0, 0.25, 0.5, 0.75, 1, 1.25, 1.5, 1.75, 2, 2.5, 3, 3.5, 4, 4.5, 5)
)
```

Arguments

r_estimate Result from `estimate_r_from_data` or numeric `r` value
r_strata Standard `r` strata breaks (default: `c(0, 0.25, 0.5, 0.75, 1, ...)`)

Value

List with stratum index, label, and suggested `r` value

launchShiny

Launch the SCP Shiny app

Description

Opens the bundled simulation-based power-analysis app in the user's default browser. The app lets the user pick a bundled pilot (or upload their own), set target power and FDR, read off the recommended sample size, explore per-gene rhythmicity, and run pathway enrichment.

Usage

```
launchShiny(..., install_deps = TRUE, species_all = FALSE)
```

Arguments

<code>...</code>	Additional arguments passed to <code>shiny::runApp()</code> .
<code>install_deps</code>	Logical; if TRUE (default) install any missing optional app packages before launching. Set FALSE to skip the check (missing features then degrade gracefully inside the app).
<code>species_all</code>	Logical; if TRUE also install the annotation databases for rat, zebrafish, Drosophila, and Arabidopsis (in addition to human + mouse) so upload gene-symbol mapping works for every supported species. Default FALSE installs only human + mouse (the common case); the others install on demand or you can pass TRUE once.

Details

On first launch the function checks for the app's optional helper packages and offers to install any that are missing (so the gene-symbol mapping, XLSX export, full-app capture, and pathway-enrichment features work out of the box). These are kept out of the package's hard dependencies so a plain `install.packages("SCP")` stays lightweight and the API works without the large Bioconductor annotation databases.

Value

Invisibly returns NULL. Called for its side effect of launching the Shiny app.

Examples

```
## Not run:
launchShiny()
launchShiny(species_all = TRUE)      # also install rat/zebrafish/fly/plant DBs
launchShiny(install_deps = FALSE)    # skip the dependency check

## End(Not run)
```

LR_deltaR2

*Likelihood Ratio Test for Delta R-squared***Description**

Likelihood Ratio Test for Delta R-squared

Usage

```
LR_deltaR2(tt1, yy1, tt2, yy2, period = 24, FN = TRUE)
```

Arguments

tt1	Time vector for condition 1
yy1	Expression vector for condition 1
tt2	Time vector for condition 2
yy2	Expression vector for condition 2
period	Period (default 24)
FN	Finite sample correction

Value

P-value for delta R2

Examples

```
set.seed(32608)
n <- 50
tt1 <- runif(n,0,24)
Amp1 <- 2
Phase1 <- 6
Offset1 <- 3
yy1 <- Amp1 * sin(2*pi/24 * (tt1 + Phase1)) + Offset1 + rnorm(n,0,1)
tt2 <- runif(n,0,24)
Amp2 <- 3
Phase2 <- 5
Offset2 <- 2
yy2 <- Amp2 * sin(2*pi/24 * (tt2 + Phase2)) + Offset2 + rnorm(n,0,1)
LR_deltaR2(tt1, yy1, tt2, yy2)
```

LR_diff

Likelihood Ratio Test for Differential Parameters

Description

Likelihood Ratio Test for Differential Parameters

Usage

```
LR_diff(time1, y1, time2, y2, period, FN = TRUE, type = "amplitude")
```

Arguments

time1	Time points for condition 1
y1	Expression values for condition 1
time2	Time points for condition 2
y2	Expression values for condition 2
period	Period
FN	Finite sample correction
type	Parameter type: "amplitude", "phase", "basal"

Value

List with p-value and parameter estimates

makeAdaptiveRStrata

Compute adaptive r-strata breakpoints with fixed bin width

Description

Generates breakpoints from 0 to $\text{ceiling}(r_{\text{max}} / \text{bin_width}) * \text{bin_width}$, stepping by `bin_width`, then appends `Inf`. Every bin width is identical, but the number of bins adapts to the empirical `r` range of the pilot data.

Usage

```
makeAdaptiveRStrata(
  bio.opts,
  bin_width = 0.25,
  r_min_pct = 0,
  r_pctile_cap = 0.99
)
```

Arguments

<code>bio.opts</code>	CircadianBioOptions with <code>amplitude</code> and <code>sigma_rhythmic</code> .
<code>bin_width</code>	Width of each <code>r</code> bin (default 0.25).
<code>r_min_pct</code>	Lower percentile of pilot <code>r</code> used as range floor (default 0 = always start from 0).
<code>r_pctile_cap</code>	Upper percentile of pilot <code>r</code> used as the range ceiling before rounding up to the nearest <code>bin_width</code> (default 0.99).

Value

Numeric vector of breakpoints starting at 0 and ending at Inf.

<code>npower</code>	<i>Find the sample size needed to achieve a target genome-wide power</i>
---------------------	--

Description

Works on output from either `runSimsSingleCohort()` or `runSimsDiff()`. Recomputes marginal power at the requested FDR from raw simulation output, then returns the smallest grid `N` that meets the target (`n_grid`) and, when `interpolate = TRUE` (the default), a linearly interpolated estimate between the two bracketing grid points (`n`).

Usage

```
npower(
  res,
  target_power = 0.8,
  fdr = 0.05,
  endpoint = NULL,
  p.adjust.method = "BH",
  interpolate = TRUE
)
```

Arguments

<code>res</code>	Output from <code>runSimsSingleCohort()</code> or <code>runSimsDiff()</code> .
<code>target_power</code>	Numeric in (0, 1). Target marginal power (default 0.80).
<code>fdr</code>	FDR threshold (default 0.05).
<code>endpoint</code>	For <code>runSimsDiff()</code> output: one of "DR", "DP", "DM". Ignored for single-cohort output.
<code>p.adjust.method</code>	Multiple-testing correction (default "BH"). Only applied for single-cohort output (differential uses pre-computed FDR arrays).
<code>interpolate</code>	Logical. If <code>TRUE</code> (default), linearly interpolates between the two adjacent grid points to return a more precise <code>n</code> estimate. Set to <code>FALSE</code> to revert to the grid-snapped <code>n_grid</code> value (backward-compatible).

Value

An object of class "npower" (a list) with:

n	Recommended sample size: n_interp when interpolate = TRUE, otherwise n_grid. NA if target power is never reached.
n_grid	Smallest sample size in the simulation grid achieving target power, or NA.
n_interp	Linearly interpolated sample size between the two bracketing grid points (a fractional N rounded up). NA when interpolate = FALSE or when target power is never reached.
power	Named numeric vector: mean power at each grid sample size.
target_power, fdr, endpoint	Echo of inputs.

one_cosinor_OLS	<i>Fit a Cosinor Model to a Single Gene by OLS</i>
-----------------	--

Description

Fits the cosinor model $y = M + A \cos(\omega t - \phi) + \epsilon$ via ordinary least squares by reparameterising as $y = M + \beta_1 \cos(\omega t) + \beta_2 \sin(\omega t) + \epsilon$. Returns mesor, amplitude, acrophase, p-value, and R².

Usage

```
one_cosinor_OLS(tod, y, period = 24, compute.phase.CI = FALSE, CI.level = 0.95)
```

Arguments

tod	Numeric vector. Sample time points (hours).
y	Numeric vector. Gene expression values (same length as tod).
period	Numeric. Rhythm period in hours (default 24).
compute.phase.CI	Logical. Reserved; not yet implemented (default FALSE).
CI.level	Numeric. Confidence level for phase CI (default 0.95).

Value

Named list with fields: M (mesor), A (amplitude), phi (acrophase in hours), pvalue (F-test p-value), R2 (coefficient of determination). Returns NA for all fields if the design matrix is singular or $n \leq 3$.

```
plot.SCPDiffResult Plot an SCPDiffResult object
```

Description

Plot an SCPDiffResult object

Usage

```
## S3 method for class 'SCPDiffResult'
plot(x, output_file = NULL, ...)
```

Arguments

x An SCPDiffResult from runDifferentialPower().

output_file Optional PDF path; NULL draws to the active device.

... Ignored.

Value

x, invisibly.

```
plot.SCPRecommendResult
Plot an SCPRecommendResult object
```

Description

Plot an SCPRecommendResult object

Usage

```
## S3 method for class 'SCPRecommendResult'
plot(x, output_file = NULL, ...)
```

Arguments

x An SCPRecommendResult from recommendDesign().

output_file Optional PDF path; NULL draws to the active device.

... Passed to the underlying plot method.

Value

x, invisibly.

```
plot.SCPSingleResult
```

Plot an SCPSingleResult object

Description

Plot an SCPSingleResult object

Usage

```
## S3 method for class 'SCPSingleResult'
plot(x, output_file = NULL, ...)
```

Arguments

x	An SCPSingleResult from runSingleCohortGrid().
output_file	Optional PDF path; NULL draws to the active device.
...	Ignored.

Value

x, invisibly.

```
plotBootstrapDesignGrid
```

Plot bootstrap design grid results

Description

Panel A: Power vs N, one line per B, with bootstrap CI ribbon. Panel B: Heatmap of mean power over (N, B) grid.

Usage

```
plotBootstrapDesignGrid(
  result,
  test_type = "DR",
  fdr_threshold = NULL,
  panels = "A",
  output_file = NULL
)
```

Arguments

result	Output from runBootstrapDesignGrid()
test_type	Test type to plot
fdr_threshold	FDR threshold (label only)
panels	Which panel(s) to draw: "A" (power vs N) and/or "B" (heatmap) (default "A").
output_file	Path for PDF output (NULL = screen)

```
plotBvsMPower =====
plot_bvsm.R,  B   vs   m   Trade-off   Plot   (Figure 3)
=====
```

Description

Produces a faceted power-vs-N figure for each dataset in the result, with one line per B value. SD bars (+/-1 SD across simulation replicates) are drawn at each N. B-invariance appears as overlapping lines.

Usage

```
plotBvsMPower (
  x,
  nsims = 30L,
  alpha2 = 0,
  fdr_power = 0.8,
  dataset_labels = NULL,
  output_file = NULL,
  width = 14,
  height = 5
)
```

Arguments

x	An SCPSingleResult from runSingleCohortGrid(), or a tidy data.frame with columns N, B, power, power_se, dataset (and optionally alpha2).
nsims	Number of simulation replicates used (to convert SE -> SD). Default 30.
alpha2	Which alpha2 value to plot (default 0 = pure cosinor).
fdr_power	Reference power level for dashed line (default 0.80).
dataset_labels	Named character vector mapping dataset codes to display labels. If NULL, uses dataset column values directly.
output_file	Path to save PDF. NULL = current graphics device.
width	PDF width in inches (default 14).
height	PDF height in inches (default 5).

Details

Plots DCP power vs N for multiple B values, demonstrating B-invariance. Input: SCPSingleResult from runSingleCohortGrid() or a tidy data.frame with columns N, B, alpha2, power, power_se. Plot B vs m Trade-off Power Grid

Value

Invisibly returns the ggplot object.

```
plotDensityResults Generate Sampling Density Plot
```

Description

Generate Sampling Density Plot

Usage

```
plotDensityResults(results_file, output_file = NULL)
```

Arguments

`results_file` Path to saved density results
`output_file` Path to save PDF figure

Value

Invisible density results

```
plotDesignComparison
```

Plot design approach comparison

Description

Panel A: Power curves, two-stage (solid blue) vs bootstrap (dashed orange + CI band). Panel B: Recommended N at target_power, bar chart comparing methods.

Usage

```
plotDesignComparison(
  comparison,
  test_type = NULL,
  target_power = NULL,
  panels = c("A", "B"),
  output_file = NULL
)
```

Arguments

`comparison` Output from compareDesignApproaches()
`test_type` Test type label
`target_power` Target power (horizontal reference line)
`panels` Which panel(s) to draw: "A" (power curves) and/or "B" (recommended-N bar chart) (default both).
`output_file` Path for PDF output (NULL = screen)

plotDiffPower	<i>Plot differential circadian power</i>
---------------	--

Description

Default layout is marginal power vs sample size only (one column per endpoint, one row per comparison). The effect-size-stratified panels (power and true-discoveries by *r*) are opt-in via `stratified = TRUE`, which restores the original 3-column-per-row layout.

Usage

```
plotDiffPower(
  res_list,
  comp_labels = names(res_list),
  endpoints = c("DR", "DP", "DM"),
  fdr_thresholds = c(0.01, 0.05, 0.1, 0.2),
  panel_fdr = 0.05,
  vline_power = 0.8,
  vline_fdr = 0.2,
  display_sizes = NULL,
  stratified = FALSE,
  r_break_width = 0.25,
  r_max = 5,
  r_display_max = NULL,
  main_title = "Differential Circadian Power Analysis",
  out_pdf = NULL,
  width = NULL,
  height = NULL
)
```

Arguments

<code>res_list</code>	Named list of length 2, each element output of <code>runSimsDiff()</code> .
<code>comp_labels</code>	Character(2), display labels for the two comparisons.
<code>endpoints</code>	Which endpoints to include (default <code>c("DR","DP","DM")</code>).
<code>fdr_thresholds</code>	FDR levels for panel A (default <code>c(0.01,0.05,0.10,0.20)</code>).
<code>panel_fdr</code>	FDR threshold used to draw the marginal power curve in the non-stratified layout (default 0.05).
<code>vline_power</code>	Horizontal reference line for target power, drawn at this value (default 0.80).
<code>vline_fdr</code>	FDR threshold marked with a reference guide on the power panels (default 0.20).
<code>display_sizes</code>	Sample sizes to show in all panels (NULL = all).
<code>stratified</code>	Logical (default FALSE). If FALSE, draws only the marginal power-vs-sample-size panel per endpoint (1 x <code>n_endpoints</code> layout per comparison). If TRUE, also draws power-by- <i>r</i> and true-discoveries-by- <i>r</i> strata panels in the original 3-column layout.
<code>r_break_width</code>	Width of <i>r</i> -strata bins (default 0.25). Ignored if <code>stratified = FALSE</code> .

<code>r_max</code>	Upper bound (in $r = A/\sigma$ units) for the r-strata breakpoints when stratified = TRUE (default 5).
<code>r_display_max</code>	Optional upper x-axis limit (r units) for the stratified panels; NULL (default) uses the full stratified range.
<code>main_title</code>	Outer plot title (default "Differential Circadian Power Analysis").
<code>out_pdf</code>	Path for PDF. If NULL, plots to current device.
<code>width</code>	PDF width in inches. Defaults adapt to stratified.
<code>height</code>	PDF height in inches. Defaults adapt to stratified.

Value

Invisibly, list of panel data per comparison/endpoint.

```
plotDRPowerStratified
```

Plot DR Power Stratified by r-Stratum

Description

Loads a saved power analysis result and produces a multi-panel plot showing detection power stratified by signal-to-noise ratio ($r = A/\sigma$) for each sample size and time-point design.

Usage

```
plotDRPowerStratified(results_file, output_file = NULL, test_name = "DR")
```

Arguments

<code>results_file</code>	Character. Path to a saved .RData results file from a power analysis run (must contain <code>dr_power_raw</code> or <code>dp_power_raw</code>).
<code>output_file</code>	Character or NULL. Path for the output PDF. If NULL, the plot is drawn to the current device.
<code>test_name</code>	Character. Label for the endpoint being plotted (default "DR"; can also be "DP").

Value

Invisibly returns NULL. Called for its side-effect of producing a plot.

plotFMMDifferential

Plot FMM Waveform Robustness: Differential Endpoints

Description

Produces a 1-row x 3-column figure showing DCP differential power (DR, DP, DM) as a function of N for multiple omega values under FMM cosinor violation.

The FMM model (Rueda, Rodriguez-Collado & Peddada, 2019, Sci Rep, doi:10.1038/s41598-019-54569-1) parameterises waveform shape via omega: omega = 1 recovers pure cosinor; omega = 0 gives a flat (arrhythmic) signal; intermediate values produce increasingly peaked waveforms. beta controls orientation (peak location) and does not affect power (beta-invariance of DCP amplitude testing).

Usage

```
plotFMMDifferential(
  x,
  nsims = 20L,
  dataset_label = NULL,
  output_file = NULL,
  width = 14,
  height = 5
)
```

Arguments

x	Data frame with columns N, omega, DR, DP, DM, SE_DR, SE_DP, SE_DM. Produced by fig4b_fmm_differential.R or any equivalent script calling simCircadianDiffFMM().
nsims	Number of simulation replicates (for SE to SD conversion).
dataset_label	Character label for the plot subtitle describing the pilot dataset used (e.g. supplied by the caller from their pilot data metadata). If NULL, an auto-generated label based on the N and omega ranges is used.
output_file	PDF path. NULL = current device.
width	PDF width in inches (default 14).
height	PDF height in inches (default 5).

Value

Invisibly returns the ggplot object.

See Also

[simCircadianDiffFMM](#), [plotFMMViolation](#)

```
plotFMMViolation  =====
                   plot_fmm.R, FMM Waveform Robustness Plots (Figures 4, 4b)
                   =====
```

Description

Produces a 2-row figure (one panel per sweep type) showing DCP power under FMM cosinor violation: row 1 sweeps omega at fixed beta; row 2 sweeps beta at fixed omega. SD bars are drawn at each N. Each panel is faceted by design type (active / passive) and SNR column (strong / moderate / weak).

The FMM model (Rueda, Rodriguez-Collado & Peddada, 2019, Sci Rep, doi:10.1038/s41598-019-54569-1) parameterises waveform shape via omega and orientation via beta. omega = 1 recovers pure cosinor, omega = 0 gives a flat (arrhythmic) signal.

Usage

```
plotFMMViolation(
  x,
  nsims = 30L,
  omega_fixed = 0.5,
  tissue_annotations = NULL,
  output_file = NULL,
  width = 16,
  height = 14
)
```

Arguments

x	Data frame with columns N, sweep_val, sweep_param, omega, beta, power, power_se, dataset, design_row, snr_col. Produced by runFMMViolationAnalysis() or the fig4_fmm_violation.R script.
nsims	Number of simulation replicates (for SE -> SD conversion).
omega_fixed	Omega value used in the beta sweep rows (for subtitle).
tissue_annotations	Optional data.frame with columns design_type, snr_col, label, N, y providing per-panel text annotations (e.g. dataset name and median r-tilde). If NULL (default), no annotations are drawn. The caller is responsible for populating this from their pilot data, hard-coded tissue names must not appear inside this package function.
output_file	PDF path. NULL = current device.
width	PDF width in inches (default 16).
height	PDF height in inches (default 14).

Details

Plotting functions for FMM-based cosinor violation analyses. Two main functions: plotFMMViolation(), single-cohort 4-row figure (Fig 4) plotFMMDifferential(), differential 3-panel figure (Fig 4b) Plot FMM Waveform Robustness: Single-Cohort

Value

Invisibly returns the combined patchwork object.

See Also

`simCircadianFMM`, `plotFMMDifferential`

```
plotFourierDeviation
```

Plot Fourier deviation power results

Description

Panel A: Heatmap of mean power at reference_n over (alpha2, alpha3) grid. Panel B: Power vs N lines, one per harmonic combination.

Usage

```
plotFourierDeviation(
  result,
  test_type = NULL,
  reference_n = NULL,
  panels = "B",
  output_file = NULL
)
```

Arguments

<code>result</code>	Output from <code>runFourierDeviationPower()</code>
<code>test_type</code>	Test type label (uses <code>result\$test_type</code> if NULL)
<code>reference_n</code>	Reference N for Panel A (uses <code>result\$reference_n</code> if NULL)
<code>panels</code>	Which panel(s) to draw: "A" (heatmap) and/or "B" (power vs N) (default "B").
<code>output_file</code>	Path for PDF output (NULL = screen)

```
plotGroundTruthComparison
```

Plot ground truth calibration results

Description

Panel A: Power vs N, oracle (black), two-stage (blue), bootstrap mean +/- CI (orange). Panel B: Per-N calibration, point estimates vs CI bars vs true value.

Usage

```
plotGroundTruthComparison(
  gt_result,
  test_type = NULL,
  target_power = 0.8,
  output_file = NULL
)
```

Arguments

gt_result	Output from runGroundTruthComparison()
test_type	Test type label (uses gt_result\$test_type if NULL)
target_power	Reference power line (default 0.80)
output_file	Path for PDF output (NULL = screen)

```
plotSingleCohortPower
```

Plot single-cohort circadian power (3 panels)

Description

Plot single-cohort circadian power (3 panels)

Usage

```
plotSingleCohortPower(
  res,
  out_pdf = NULL,
  title = "",
  panel_a_res = NULL,
  panels = c("A", "B", "C"),
  fdr_thresholds = c(0.01, 0.05, 0.1, 0.2),
  panel_fdr = 0.05,
  vline_power = 0.8,
  vline_fdr = 0.2,
  fdr = NULL,
  p.adjust.method = "BH",
  reference_n = NULL,
  display_sizes = NULL,
  r_max = 5,
  cex_main = 1.5,
  cex_lab = 1.6,
  cex_axis = 1.45,
  line_lwd = 2,
  pt_cex = 0.95,
  legend_cex = NULL,
  title_cex = 1.65,
  oma_top = 2.4,
  legend_pos = "topleft",
  se_cap = 0.3,
```

```

vertical = FALSE,
panel_c_legend = FALSE,
width = 15,
height = 5.5
)

```

Arguments

<code>res</code>	Output from <code>runSimsSingleCohort()</code> .
<code>out_pdf</code>	Path for PDF output. If NULL, plots to current device.
<code>title</code>	Overall figure title (default empty).
<code>panel_a_res</code>	Optional alternate result object used for the Panel A marginal-power computation only. If NULL (default), Panel A is computed from <code>res</code> . Useful when the marginal summary should reflect a different empirical assumption than the effect-size-stratified panels.
<code>panels</code>	Character vector, which of panels "A" (marginal power vs n), "B" (true discoveries by r-stratum), "C" (stratified power by r-stratum) to draw (default all three).
<code>fdr_thresholds</code>	FDR levels for panel A (default <code>c(0.01,0.05,0.10,0.20)</code>).
<code>panel_fdr</code>	FDR threshold highlighted in panels B/C (default 0.05).
<code>vline_power</code>	Horizontal reference line for target power (default 0.80).
<code>vline_fdr</code>	FDR threshold marked with a reference guide (default 0.20).
<code>fdr</code>	Single-value alias that sets both <code>panel_fdr</code> and <code>vline_fdr</code> at once (back-compat convenience; default NULL).
<code>p.adjust.method</code>	Correction method (default "BH").
<code>reference_n</code>	Reference sample size for annotations (default NULL = none).
<code>display_sizes</code>	Sample sizes to show in all panels (NULL = all).
<code>r_max</code>	Upper bound ($r = A/\sigma$ units) for the r-strata breakpoints; NULL adapts to the 95th percentile of observed r (default 5).
<code>cex_main</code>	Title character expansion (default 1.50).
<code>cex_lab</code>	Axis-label character expansion (default 1.60).
<code>cex_axis</code>	Axis-tick character expansion (default 1.45).
<code>line_lwd</code>	Line width for power curves (default 2).
<code>pt_cex</code>	Point character expansion for plotted markers (default 0.95).
<code>legend_cex</code>	Legend text character expansion; NULL (default) picks a size automatically.
<code>title_cex</code>	Outer-title character expansion (default 1.65).
<code>oma_top</code>	Outer margin (lines) reserved above the panels for the title (default 2.4).
<code>legend_pos</code>	Legend position keyword passed to <code>legend()</code> (default "topleft").
<code>se_cap</code>	Maximum half-width (power units) drawn for standard-error bars, to keep long bars from overplotting neighboring panels (default 0.3).
<code>vertical</code>	Logical; if TRUE, stack panels vertically instead of the default horizontal 1 x <code>length(panels)</code> layout (default FALSE).

panel_c_legend	Logical; if TRUE, also draw the legend on panel C (default FALSE; panel A's legend is normally sufficient).
width	PDF width in inches (default 15).
height	PDF height in inches (default 5.5).

Value

Invisibly returns list of data used per panel.

plot_power_curves *Plot Power Curves*

Description

Visualize power vs sample size for different methods

Usage

```
plot_power_curves(power_results, target_power = 0.8)
```

Arguments

power_results Results from power_single_condition
target_power Target power level (default 0.8)

Value

ggplot object

plot_stratified_power
Plot Stratified Power

Description

Bar plot of power by effect size stratum

Usage

```
plot_stratified_power(stratified_results)
```

Arguments

stratified_results Results from stratified_power

Value

ggplot object

```
plot_two_group_heatmap
```

Plot Two-Group Power Heatmap

Description

Heatmap of power for different (n_A, n_B) combinations

Usage

```
plot_two_group_heatmap(power_matrix, n_A_range, n_B_range)
```

Arguments

`power_matrix` Power matrix from `power_two_group`

`n_A_range` n_A values

`n_B_range` n_B values

Value

ggplot object

```
power_assessment
```

Power Assessment Summary

Description

Power assessment combining marginal, stratified, and targeted metrics.

Usage

```
power_assessment(pvals, ground_truth, effect_sizes, alpha = 0.05)
```

Arguments

`pvals` P-values

`ground_truth` True positive indicator

`effect_sizes` Effect sizes

`alpha` Significance threshold

Value

List of overall, stratified, and targeted power metrics.

```
prepCircadianData
```

Prepare expression data for circadian power analysis

Description

Converts raw expression data into the standard (matrix, times) format expected by `estCircadianParam()`, `estCircadianParamTwoGroup()`, and `runBootstrapDesignGrid()`.

The pipeline always expects:

- data, numeric matrix, **genes x samples**, log2-scale
- times, numeric vector, hours in [0, 24), same length as `ncol(data)`

This function handles the three most common raw formats:

"counts" Raw integer counts (e.g. `featureCounts`, `STAR`). Converts to $\log_2(\text{CPM} + 1)$: `log2(sweep(x, 2, colSums(x), "/") * 1e6 + 1)`.

"cpm" Raw CPM values (e.g. `edgeR cpm()`, baboon data). Converts to $\log_2(\text{CPM} + 1)$.

"log2" Already log2-scale (e.g. GSE54651, Seney ACC). Wraps with `data.matrix()` only; no numeric transformation.

Usage

```
prepCircadianData(
  expr,
  times,
  input_type = c("counts", "cpm", "log2"),
  pheno = NULL,
  sample_col = NULL,
  time_col = NULL,
  period = 24,
  verbose = TRUE
)
```

Arguments

<code>expr</code>	A matrix, data.frame, or file path (CSV/TSV) of expression values. Rows = genes, columns = samples. Row names used as gene IDs.
<code>times</code>	Numeric vector of time-of-day values (hours, 0-24) for each sample, OR a character column name in <code>pheno</code> to extract times from.
<code>input_type</code>	One of "counts", "cpm", or "log2". Default "counts".
<code>pheno</code>	Optional data.frame of sample metadata. Must have the same column order as <code>expr</code> if <code>times</code> is a column name string.
<code>sample_col</code>	Optional: name of the column in <code>pheno</code> whose values match <code>colnames(expr)</code> for alignment. If NULL, assumes columns are already in the same order as rows of <code>pheno</code> .
<code>time_col</code>	Name of the TOD column in <code>pheno</code> (used when <code>times</code> is a column name string).
<code>period</code>	Circadian period in hours. Default 24. Used only for validation.
<code>verbose</code>	Print a short summary of what was done. Default TRUE.

Value

A list with:

<code>data</code>	Numeric matrix (genes x samples), log2-scale
<code>times</code>	Numeric vector of hours (0-24), length == ncol(data)
<code>n_genes</code>	Number of genes
<code>n_samples</code>	Number of samples
<code>input_type</code>	The <code>input_type</code> used

Examples

```
## Not run:
# From raw counts matrix
prep <- prepCircadianData(counts_matrix, times = tod_vector, input_type = "counts")
bio <- estCircadianParam(prepare$prep, prepare$times)

# From data.frame with metadata
prep <- prepCircadianData(
  expr      = expr_df,
  times     = "TOD",
  input_type = "log2",
  pheno     = pheno_df,
  sample_col = "SampleID"
)

# Baboon raw CPM (load gives a data.frame)
prep <- prepCircadianData(baboon_df, times = tod_vector, input_type = "cpm")

## End(Not run)
```

```
print.CircadianAnalysisOptions
```

Print a CircadianAnalysisOptions object

Description

Print a CircadianAnalysisOptions object

Usage

```
## S3 method for class 'CircadianAnalysisOptions'
print(x, ...)
```

Arguments

<code>x</code>	A CircadianAnalysisOptions object.
<code>...</code>	Ignored.

Value

`x`, invisibly.

```
print.CircadianBioOptions
```

Print a CircadianBioOptions object

Description

Print a CircadianBioOptions object

Usage

```
## S3 method for class 'CircadianBioOptions'
print(x, ...)
```

Arguments

x	A CircadianBioOptions object.
...	Ignored.

Value

x, invisibly.

```
print.CircadianBootstrapOptions
```

Print a CircadianBootstrapOptions object

Description

Print a CircadianBootstrapOptions object

Usage

```
## S3 method for class 'CircadianBootstrapOptions'
print(x, ...)
```

Arguments

x	A CircadianBootstrapOptions object.
...	Ignored.

Value

x, invisibly.

```
print.CircadianDesignOptions
      Print a CircadianDesignOptions object
```

Description

Print a CircadianDesignOptions object

Usage

```
## S3 method for class 'CircadianDesignOptions'
print(x, ...)
```

Arguments

x	A CircadianDesignOptions object.
...	Ignored.

Value

x, invisibly.

```
print.estimate_r_from_data
      Print method for estimate_r_from_data
```

Description

Print method for estimate_r_from_data

Usage

```
## S3 method for class 'estimate_r_from_data'
print(x, ...)
```

Arguments

x	Result from estimate_r_from_data
...	Additional arguments (ignored)

Value

x, invisibly.

print.npower	<i>Print an npower result</i>
--------------	-------------------------------

Description

Print an npower result

Usage

```
## S3 method for class 'npower'
print(x, ...)
```

Arguments

x	An npower object (recommended n and its power curve).
...	Ignored.

Value

x, invisibly.

print.SCPDiffResult	<i>Print an SCPDiffResult object</i>
---------------------	--------------------------------------

Description

Print an SCPDiffResult object

Usage

```
## S3 method for class 'SCPDiffResult'
print(x, ...)
```

Arguments

x	An SCPDiffResult from runDifferentialPower().
...	Ignored.

Value

x, invisibly.

```
print.SCPRecommendResult
```

Print an SCPRecommendResult object

Description

Print an SCPRecommendResult object

Usage

```
## S3 method for class 'SCPRecommendResult'
print(x, ...)
```

Arguments

x	An SCPRecommendResult from recommendDesign().
...	Ignored.

Value

x, invisibly.

```
print.SCPSingleResult
```

Print an SCPSingleResult object

Description

Print an SCPSingleResult object

Usage

```
## S3 method for class 'SCPSingleResult'
print(x, ...)
```

Arguments

x	An SCPSingleResult from runSingleCohortGrid().
...	Ignored.

Value

x, invisibly.

```
printDensitySummary
```

Print Density Analysis Summary Table

Description

Prints a formatted text table of power, true discoveries, false discoveries, and false discovery cost from a density analysis result, stratified by sample size and number of time points.

Usage

```
printDensitySummary(density_results)
```

Arguments

```
density_results
```

List. Output of `runSimsDensity()` containing `n_subjects`, `n_time_points`, `power_avg`, `TD_avg`, `FD_avg`, `FDC_avg`, and `test_type`.

Value

Invisibly returns `NULL`. Called for its side-effect of printing to the console.

```
printMethodGuidance
```

Print B vs m method guidance table

Description

Prints a table explaining how each detection method responds to B (time points) vs m (replicates per ZT). Called automatically by `recommendDesign()` and `runSingleCohortPower()` when `verbose=TRUE`.

Usage

```
printMethodGuidance(methods = c("DCP", "JTK", "RAIN", "MH"), verbose = TRUE)
```

Arguments

<code>methods</code>	Character vector of methods to include
<code>verbose</code>	Print to console (default TRUE)

Value

Invisible `data.frame[method, recommended_B, B_vs_m, reason]`

```
rayleigh_test_circular
```

Rayleigh test for uniform circular distribution

Description

Rayleigh test for uniform circular distribution

Usage

```
rayleigh_test_circular(angles, period = 24)
```

Arguments

angles	Angles (in hours)
period	Period (default 24)

Value

List with statistic and p-value

```
recommendDesign
```

B vs m design study – analytical + simulation

Description

Full B vs m study orchestrator. Three sequential steps:

Usage

```
recommendDesign(
  bio.opts,
  design.opts,
  analysis.opts,
  methods = c("DCP", "JTK", "RAIN", "MH"),
  target_power = 0.8,
  mode = c("single", "differential"),
  run_simulation = TRUE,
  prior_result = NULL,
  alpha2 = 0,
  alpha3 = 0,
  mc.cores = 1L,
  plot = TRUE,
  output_file = NULL,
  verbose = TRUE
)
```

Arguments

<code>bio.opts</code>	CircadianBioOptions from <code>estCircadianParam()</code> or <code>estCircadianParamTwoGroup()</code>
<code>design.opts</code>	CircadianDesignOptions with <code>sample_sizes</code> and <code>B_values</code>
<code>analysis.opts</code>	CircadianAnalysisOptions
<code>methods</code>	Detection methods to compare. Single-cohort: <code>c("DCP","JTK","RAIN","MH")</code> . Differential: <code>c("DCP","CircaCompare","LimoRhyde","DODR")</code> .
<code>target_power</code>	Target power for recommendation (default 0.80)
<code>mode</code>	"single" or "differential" (default "single")
<code>run_simulation</code>	TRUE = run full simulation after analytical step. FALSE = analytical (CircaPower) only.
<code>prior_result</code>	A previous <code>runSingleCohortGrid()/runDifferentialPower()</code> result to reuse instead of re-running simulation.
<code>alpha2</code>	2nd-harmonic deviation (scalar or vector)
<code>alpha3</code>	3rd-harmonic deviation (scalar or vector)
<code>mc.cores</code>	Parallel cores for simulation step
<code>plot</code>	Auto-plot final recommendation
<code>output_file</code>	PDF path (NULL = screen)
<code>verbose</code>	Print progress

Details

1. Print method guidance table (`printMethodGuidance`) 2. Analytical: CircaPower closed-form power at each N (DCP; B-invariant). For JTK/RAIN/MH no closed form exists – simulation is required. 3. Simulation: run `runSingleCohortPower()` or `runDifferentialPower()`, or absorb a `prior_result` from a previous call to skip re-running.

Value

SCPRRecommendResult with: `$guidance` – data.frame: method guidance table `$analytical_df` – data.frame[method, N, power_analytical] (DCP only) `$simulation` – `runSingleCohortGrid()` or `runDifferentialPower()` result (NULL if not run) `$recommendation` – data.frame[method, optimal_B, n_target, note]

replotDRPower

DR Power Analysis Plotting Functions

Description

Functions for generating stratified power plots from saved DR simulation results Quick Replot DR Power (convenience wrapper)

Usage

```
replotDRPower(
  test_name = "DR",
  results_dir = "output/dr_power_stratified",
  output_file = NULL
)
```

Arguments

```
test_name      "DR" or "DP"
results_dir    Directory containing results files
output_file    Path to save PDF (optional)
```

Value

Invisible list with calculated power statistics

```
runBootstrapDesignGrid
      Run bootstrap design grid simulation
```

Description

Estimates uncertainty in power across (N, B) design configurations by bootstrapping parameter sets from pilot data.

Usage

```
runBootstrapDesignGrid(
  pilot_data,
  pilot_times,
  boot.opts,
  analysis.opts,
  bio_diff.opts,
  pilot_data_2 = NULL,
  pilot_times_2 = NULL,
  mode = NULL,
  methods = "DCP",
  test_types = c("DR", "DP", "DM"),
  alpha2 = 0,
  alpha3 = 0,
  min_rhythm_pval = 0.01,
  resample = c("subject", "gene"),
  verbose = TRUE,
  mc.cores = 1L
)
```

Arguments

<code>pilot_data</code>	Genes x samples matrix (pilot experiment).
<code>pilot_times</code>	Numeric vector of sample time points, <code>length = ncol(pilot_data)</code> .
<code>boot.opts</code>	<code>CircadianBootstrapOptions</code> from <code>CircadianBootstrapOptions()</code> . Must set <code>B_values</code> to a single value for passive designs.
<code>analysis.opts</code>	<code>CircadianAnalysisOptions</code> .
<code>bio_diff.opts</code>	<code>CircadianBioOptions</code> from <code>estCircadianParam()</code> . Required even in <code>mode = "single"</code> , differential proportion fields are read from here.
<code>pilot_data_2</code>	Optional group-2 pilot matrix for <code>mode = "differential"</code> .
<code>pilot_times_2</code>	Optional group-2 pilot times.
<code>mode</code>	"single" or "differential".
<code>methods</code>	Detection method(s): "DCP" (default), "JTK", "RAIN".
<code>test_types</code>	Endpoints for differential mode: any of "DR", "DP", "DM".
<code>alpha2</code>	Second-harmonic coefficient (default 0).
<code>alpha3</code>	Third-harmonic coefficient (default 0).
<code>min_rhythm_pval</code>	P-value threshold for pilot rhythmicity (default 0.01).
<code>resample</code>	Resampling scheme for the outer bootstrap. "subject" (default) is the Efron nonparametric bootstrap over pilot subjects: each draw resamples subject columns with replacement (carrying their collection times) and refits the pilot summary, matching the Section 2.3 description. "gene" is the legacy scheme that fits once and resamples gene rows of the fit; retained for backward compatibility and diagnostics only.
<code>verbose</code>	Print progress (default TRUE).
<code>mc.cores</code>	Parallel cores for inner simulation loop (default 1).

Value

Named list with 15 elements:

<code>N_values</code>	Candidate sample sizes tested.
<code>B_values</code>	Candidate time-point counts tested.
<code>m_matrix</code>	Matrix of total samples ($N \times B$).
<code>nboot</code>	Number of bootstrap replicates.
<code>design</code>	Design type ("active" or "passive").
<code>test_types</code>	Endpoints evaluated.
<code>fdr_threshold</code>	FDR threshold used.
<code>boot_power</code>	4-D array [$nboot \times n_N \times n_B \times n_tests$].
<code>power_mean</code>	Mean power array [$n_N \times n_B \times n_tests$].
<code>power_se</code>	Bootstrap SE array, same shape as <code>power_mean</code> .
<code>power_ci_lo</code>	2.5th percentile CI array.
<code>power_ci_hi</code>	97.5th percentile CI array.
<code>optimal_B</code>	Integer vector (length n_N): B with highest mean power at each N.
<code>optimal_B_ci_lo</code>	Bootstrap 2.5th percentile of optimal B.
<code>optimal_B_ci_hi</code>	Bootstrap 97.5th percentile of optimal B.

runDifferentialPower

Run two-group differential circadian power analysis

Description

Sweeps $N \times \alpha_2 \times \alpha_3 \times \text{method} \times \text{test_type}$ and returns a tidy power table. Automatically calls `plot()` and `npower()` unless `plot=FALSE`.

Usage

```
runDifferentialPower(
  bio.opts,
  design.opts,
  analysis.opts,
  methods = "DCP",
  test_types = c("DR", "DP", "DM"),
  alpha2 = 0,
  alpha3 = 0,
  mc.cores = 1L,
  plot = TRUE,
  output_file = NULL,
  verbose = TRUE
)
```

Arguments

<code>bio.opts</code>	CircadianBioOptions from <code>estCircadianParamTwoGroup()</code>
<code>design.opts</code>	CircadianDesignOptions (sample_sizes, design, cts)
<code>analysis.opts</code>	CircadianAnalysisOptions
<code>methods</code>	Any of "DCP", "CircaCompare"
<code>test_types</code>	Any of "DR", "DP", "DM" (silently NA if method lacks support)
<code>alpha2</code>	Scalar or vector swept for both groups
<code>alpha3</code>	Scalar or vector swept for both groups
<code>mc.cores</code>	Parallel cores
<code>plot</code>	Auto-plot on completion
<code>output_file</code>	PDF path (NULL = screen)
<code>verbose</code>	Print progress

Value

List (class `SCPDiffResult`) from `runSimsDiff()`: `pval_DR`, `fdr_DR`, `pval_DP`, `fdr_DP`, `pval_DM`, `fdr_DM`—3-D arrays [`ngenes` $\times$ `n_sizes` $\times$ `nsims`]; plus `diff_type`, `effectsize`, `sample_sizes`, `nsims`. Pass to `plotDiffPower()` or `npower(..., endpoint="DR")`.

```
runFourierDeviationPower
```

Fourier Deviation Simulation

Description

Tests how power degrades when the true waveform has higher harmonics (2nd, 3rd) that DCP's cosinor model doesn't fit.

Usage

```
runFourierDeviationPower(
  bio.opts,
  design.opts,
  analysis.opts,
  harmonic_grid = NULL,
  test_type = "DR",
  verbose = TRUE,
  mc.cores = 1L
)
```

Arguments

<code>bio.opts</code>	CircadianBioOptions
<code>design.opts</code>	CircadianDesignOptions
<code>analysis.opts</code>	CircadianAnalysisOptions
<code>harmonic_grid</code>	data.frame with columns alpha2, alpha3 (default: 4x3 grid)
<code>test_type</code>	"DR", "DP", or "DM"
<code>verbose</code>	Print progress
<code>mc.cores</code>	Parallel cores (default 1).

Details

The cosinor model assumes: $y(t) = M + A \cdot \cos(\omega t - \omega \phi) + \epsilon$

But the true waveform may be: $y(t) = M + A \cdot [\cos(\omega t - \omega \phi) + \alpha_2 \cdot \cos(2\omega t - \omega \phi) + \alpha_3 \cdot \cos(3\omega t - \omega \phi)] + \epsilon$

where α_2 , α_3 are relative harmonic amplitudes. This module sweeps over (α_2 , α_3) combinations and measures resulting power degradation. Run Fourier deviation power analysis

Measures how power degrades as higher harmonics are added to the true waveform while the detection model remains a pure cosinor (fundamental only).

Value

List with power arrays and summaries across harmonic levels

```
runGroundTruthComparison
```

Ground truth calibration: compare two-stage vs bootstrap when truth is known

Description

Generates synthetic pilot data from known parameters, then evaluates how well two-stage (point estimate) and bootstrap (distribution) approaches recover the true power curve.

Usage

```
runGroundTruthComparison(
  true_bio.opts,
  design.opts,
  analysis.opts,
  n_pilot,
  pilot_times,
  nboot = 50,
  nsims_inner = 10,
  test_type = "DR",
  seed = 42,
  verbose = TRUE
)
```

Arguments

<code>true_bio.opts</code>	CircadianBioOptions, the known ground truth biology (includes both base params AND differential params)
<code>design.opts</code>	CircadianDesignOptions, N values and design to evaluate
<code>analysis.opts</code>	CircadianAnalysisOptions
<code>n_pilot</code>	Number of synthetic pilot subjects
<code>pilot_times</code>	TOD distribution for pilot time sampling
<code>nboot</code>	Bootstrap draws for uncertainty quantification
<code>nsims_inner</code>	Simulations per bootstrap draw
<code>test_type</code>	"DR", "DP", or "DM"
<code>seed</code>	Random seed
<code>verbose</code>	Print progress

Value

List with `true_power`, two-stage power, and bootstrap power +/- CI

```
runPhaseShiftAnalysis
```

Sweep over phase shift magnitudes AND sample sizes.

Description

Sweep over phase shift magnitudes AND sample sizes.

Usage

```
runPhaseShiftAnalysis(
  bio.opts,
  design.opts,
  analysis.opts,
  prop_DP = 0.15,
  amp_diff = c(0.5, 2),
  mc.cores = 1L
)
```

Arguments

bio.opts	CircadianBioOptions (base; prop_DP/phase_diff will be overridden)
design.opts	CircadianDesignOptions
analysis.opts	CircadianAnalysisOptions (phase_shifts read from here)
prop_DP	Proportion of DP genes for each scenario (default 0.15)
amp_diff	Amplitude ratio range for DP genes (default c(0.5, 2))
mc.cores	Parallel cores (default 1).

Value

List with 4D arrays [phase, size, stratum, sim] compatible with plotPhaseShiftWithSE()

```
runPowerAnalysis
```

Run power analysis with $r = A/\sigma$ stratification.

Description

Loops over sample sizes, calls runSimsDiff() for each, and computes stratified power / TD / FD at FDR 5

Usage

```
runPowerAnalysis(
  bio.opts,
  design.opts,
  analysis.opts,
  test_type = "DR",
  verbose = TRUE
)
```

Arguments

<code>bio.opts</code>	CircadianBioOptions
<code>design.opts</code>	CircadianDesignOptions
<code>analysis.opts</code>	CircadianAnalysisOptions
<code>test_type</code>	"DR" or "DP"
<code>verbose</code>	Print progress (default TRUE).

Value

List compatible with `plotWithSE()`

<code>runSCPValidation</code>	<i>Run the SCP validation suite</i>
-------------------------------	-------------------------------------

Description

Run the SCP validation suite

Usage

```
runSCPValidation(nsims = 100L, n_test = 24L, alpha = 0.05, seed = 42L)
```

Arguments

<code>nsims</code>	Simulation replicates per validation (default 100).
<code>n_test</code>	Sample size for single-n validations (default 24).
<code>alpha</code>	Nominal significance level (default 0.05).
<code>seed</code>	RNG seed for reproducibility (default 42).

Value

Invisible list of the six validation result tables (v1 to v6).

<code>runSimsDiff</code>	<i>Low-level differential power simulation engine</i>
--------------------------	---

Description

Preferred interface: `runDifferentialPower(bio.opts, design.opts, analysis.opts)`.
 This function supports both the config-object API (primary) and a legacy flat-argument API (retained for back-compatibility).

Usage

```
runSimsDiff(
  sample_sizes = c(12, 24, 36),
  nsims = 50,
  ngenes = 5000,
  prop_rhythmic = 0.25,
  prop_DR = 0.1,
  prop_DP = 0.1,
  phase_diff = NULL,
  amp_diff = NULL,
  design = c("active", "passive"),
  cts = NULL,
  test_types = c("DR", "DP", "DM"),
  verbose = TRUE,
  harmonics = NULL,
  mc.cores = 1L,
  design.opts = NULL,
  analysis.opts = NULL
)
```

Arguments

<code>sample_sizes</code>	CircadianBioOptions (primary) or numeric vector of per-group sample sizes (legacy flat API).
<code>nsims</code>	CircadianDesignOptions (primary) or number of simulations (legacy).
<code>ngenes</code>	CircadianAnalysisOptions (primary) or number of genes (legacy).
<code>prop_rhythmic</code>	Overall proportion of rhythmic genes (legacy only).
<code>prop_DR</code>	Proportion with differential rhythmicity (legacy only).
<code>prop_DP</code>	Proportion with differential phase (legacy only).
<code>phase_diff</code>	Phase-shift range in hours for DP genes (legacy only).
<code>amp_diff</code>	Unused; retained for interface compatibility.
<code>design</code>	"active" or "passive" (legacy only).
<code>cts</code>	TOD distribution for passive design (legacy only).
<code>test_types</code>	Which endpoints to evaluate: any of "DR", "DP", "DM".
<code>verbose</code>	Print progress.
<code>harmonics</code>	Numeric length-2 <code>c(alpha2, alpha3)</code> harmonic coefficients for non-cosinor signal generation (legacy only; default NULL, equivalent to <code>c(0, 0)</code>).
<code>mc.cores</code>	Parallel cores.
<code>design.opts</code>	CircadianDesignOptions (used with legacy positional first arg).
<code>analysis.opts</code>	CircadianAnalysisOptions.

Value

List (class `SCPDiffResult`) with fields: `pval_DR`, `fdr_DR`, `pval_DP`, `fdr_DP`, `pval_DM`, `fdr_DM`—arrays [`ngenes` × `n_sizes` × `nsims`]; `diff_type`, `effectsize`, `sample_sizes`, `nsims`. Pass directly to `plotDiffPower()` or `npower()`.

See Also

[runDifferentialPower](#) (user-facing wrapper)

```
runSimsSingleCohort
```

Simulation-Based Power for Single-Cohort Rhythmicity Detection

Description

Extends the closed-form CircaPower approach by running full simulations: for each sample size and simulation replicate, generates data from the empirical pilot parameter distributions, applies the cosinor F-test per gene, applies BH correction, and aggregates empirical power and FDR.

Usage

```
runSimsSingleCohort (
  bio.opts,
  design.opts,
  analysis.opts,
  method = "cosinor",
  K = 1L,
  harmonics = c(0, 0),
  verbose = TRUE,
  mc.cores = 1L,
  progress = NULL
)
```

Arguments

bio.opts	CircadianBioOptions – pilot parameter distributions. Build with <code>estCircadianParam()</code> .
design.opts	CircadianDesignOptions – sample sizes, nsims, design.
analysis.opts	CircadianAnalysisOptions – alpha, p.adjust.method.
method	Detection method: "DCP" (1-harmonic cosinor F-test), "JTK", "RAIN", "MH" (multi-harmonic), or "FMM" (K-harmonic F-test motivated by the FMM Fourier expansion; default detector for non-cosinor signals).
K	Integer. Number of harmonics for method = "FMM" (default 2). K=1 is equivalent to DCP; K=2 captures the 1st and 2nd Fourier harmonics; K=3+ for very sharp peaks.
harmonics	Numeric length-2: <code>c(alpha2, alpha3)</code> harmonic coefficients (default <code>c(0, 0)</code>).
verbose	Print progress (default TRUE).
mc.cores	Parallel cores (default 1).
progress	Optional callback function(<code>sample_size_index, n_sample_sizes</code>) invoked after each sample size completes (used by the Shiny app to advance its progress bar). NULL (default) disables the callback.

Details

Simulation-based alternative to the closed-form CircaPower formula: works for any design (active or passive) and any pilot parameter distribution.

Value

List with:

```

marginal_power      Matrix [sample_sizes x nsims]
marginal_FDR        Matrix [sample_sizes x nsims]
marginal_TD         Matrix [sample_sizes x nsims]
marginal_FD         Matrix [sample_sizes x nsims]
marginal_alpha      Matrix [sample_sizes x nsims] – empirical type-I error
pvalues             Array [sample_sizes x ngenes x nsims] – raw p-values
r_values_list       List[[sample_size]][[sim]] – per-gene A/sigma
strat_power         Array [sample_sizes x n_strata x nsims]
strat_TD            Array [sample_sizes x n_strata x nsims]
strat_FD            Array [sample_sizes x n_strata x nsims]
strat_n_targets     Array [sample_sizes x n_strata x nsims]
strat_n_nulls       Array [sample_sizes x n_strata x nsims]
strat_FDR           Array [sample_sizes x n_strata x nsims]
strat_alpha         Array [sample_sizes x n_strata x nsims]
r_strata            Stratum breakpoints (from analysis.opts)
strata_labels       Stratum labels
n0_circapower       CircaPower n80 estimate from pilot median r
sample_sizes        Vector of sample sizes
nsims               Number of simulation replicates

```

Examples

```

## Not run:
bio <- estCircadianParam(expr, times)
dopt <- CircadianDesignOptions(sample_sizes = c(20, 40, 80), nsims = 50)
aopt <- CircadianAnalysisOptions(alpha = 0.05)
res <- runSimsSingleCohort(bio, dopt, aopt)
rowMeans(res$marginal_power) # mean power at each N

## End(Not run)

```

```
runSingleCohortGrid
```

Single-cohort B-versus-m power grid (active designs)

Description

Sweeps sample size, number of time points, and detection method to produce the power grid used by `recommendDesign`. Active (balanced) designs only.

Usage

```
runSingleCohortGrid(
  bio.opts,
  design.opts,
  analysis.opts,
  methods = "DCP",
  alpha2 = 0,
  alpha3 = 0,
  mc.cores = 1L,
  plot = FALSE,
  output_file = NULL,
  verbose = TRUE,
  target_power = 0.8
)
```

Arguments

<code>bio.opts</code>	Pilot summary (<code>CircadianBioOptions</code>).
<code>design.opts</code>	Design options (<code>CircadianDesignOptions</code>).
<code>analysis.opts</code>	Analysis options (<code>CircadianAnalysisOptions</code>).
<code>methods</code>	Detection method(s); default "DCP".
<code>alpha2, alpha3</code>	Second- and third-harmonic generative coefficients (default 0).
<code>mc.cores</code>	Number of cores.
<code>plot</code>	Whether to draw the grid.
<code>output_file</code>	Optional output path for the plot.
<code>verbose</code>	Print progress.
<code>target_power</code>	Power threshold used to compute <code>n80_df</code> (despite the name, the interpolated sample size is for this threshold, not hardcoded to 80%). Default 0.80.

Value

An `SCPSingleResult` with the power grid and N80 table.

runSingleCohortPower

Run single-cohort rhythmicity power analysis

Description

Sweeps $N \times B \times \alpha_2 \times \alpha_3 \times \text{method}$ and returns a tidy power table. B values come from `design.opts$B_values`; if `NULL`, B is inferred from `cts`. Automatically calls `plot()` and `npower()` unless `plot=FALSE`.

Usage

```
runSingleCohortPower(
  bio.opts,
  design.opts,
  analysis.opts,
  methods = "DCP",
  alpha2 = 0,
  alpha3 = 0,
  mc.cores = 1L,
  plot = TRUE,
  output_file = NULL,
  verbose = TRUE
)
```

Arguments

<code>bio.opts</code>	CircadianBioOptions from <code>estCircadianParam()</code>
<code>design.opts</code>	CircadianDesignOptions (<code>sample_sizes</code> , <code>B_values</code> , <code>cts</code> , <code>design</code>)
<code>analysis.opts</code>	CircadianAnalysisOptions
<code>methods</code>	Character vector: any of "DCP", "JTK", "RAIN", "MH"
<code>alpha2</code>	Scalar or vector of 2nd-harmonic amplitudes to sweep
<code>alpha3</code>	Scalar or vector of 3rd-harmonic amplitudes to sweep
<code>mc.cores</code>	Parallel cores
<code>plot</code>	Auto-plot on completion
<code>output_file</code>	PDF path for auto-plot (<code>NULL</code> = screen)
<code>verbose</code>	Print progress

Value

Rich list (from `runSimsSingleCohort`): `$marginal_power` [$N \times \text{nsims}$], `$strat_power` [$N \times r_{\text{strata}} \times \text{nsims}$], `$pvalues` [$N \times \text{genes} \times \text{nsims}$], `$sample_sizes`, `$r_values_list`. Pass directly to `plotSingleCohortPower()`.

runTwoStagePower *Design Comparison: Two-Stage vs Bootstrap-Simulation Power*

Description

Compares two approaches to pilot-informed power analysis for a FIXED design:

Usage

```
runTwoStagePower (
  pilot_data,
  pilot_times,
  design.opts,
  analysis.opts,
  bio_diff.opts,
  min_rhythm_pval = 0.01,
  test_type = "DR",
  verbose = TRUE,
  mc.cores = 1L
)
```

Arguments

pilot_data	Genes x samples matrix
pilot_times	Sample time points (length = ncol(pilot_data))
design.opts	CircadianDesignOptions
analysis.opts	CircadianAnalysisOptions
bio_diff.opts	CircadianBioOptions (for differential params only)
min_rhythm_pval	Threshold for calling rhythmic genes in pilot
test_type	"DR", "DP", or "DM"
verbose	Print progress
mc.cores	Parallel cores (default 1).

Details

1. Two-stage (point estimate): Estimate circadian parameters from pilot once, simulate with those fixed estimates -> one power curve (no uncertainty).
2. Bootstrap simulation: Resample pilot gene rows (nboot times) to get a distribution of parameter sets, simulate for each -> power +/- CI.

The comparison isolates pilot-estimation uncertainty: both approaches use the same design (same B and m); the only difference is whether the parameter distribution is treated as fixed or as an uncertain quantity.

IMPORTANT: Pass bootstrap results from a single fixed B value. Averaging over multiple candidate B values produces a quantity with no actionable scientific meaning, because the user must choose

one specific design, not the mean of several designs with different B/m splits. Run two-stage power analysis (point estimate approach)

Estimates circadian parameters from pilot data once, then runs power simulations using those point-estimate parameters.

Value

List with marginal_power matrix [n_sizes x nsims] and sample_sizes

run_DCP_pipeline	<i>Run full DCP differential analysis pipeline</i>
------------------	--

Description

Uses the complete DiffCircadian pipeline with TOJR, hierarchical testing, and proper LR tests. This is the RIGOROUS approach.

Usage

```
run_DCP_pipeline(
  expr1,
  expr2,
  times1,
  times2,
  gene_names = NULL,
  period = 24,
  alpha = 0.05,
  test_DR = TRUE,
  test_DP = TRUE
)
```

Arguments

expr1	Expression matrix for group 1 (genes x samples)
expr2	Expression matrix for group 2 (genes x samples)
times1	Time points for group 1
times2	Time points for group 2
gene_names	Gene names (optional)
period	Period (default 24)
alpha	Significance threshold
test_DR	Test differential rhythmicity?
test_DP	Test differential phase?

Value

List with p-values for requested tests

```
sampleTimesFromDist
```

Sample Time Points from an Empirical TOD Distribution

Description

Draws n time-of-day (TOD) values from a circular kernel density estimate of a pilot TOD distribution. Handles wrap-around at 0h/24h using a tripled-data trick so boundary contributions are preserved.

Usage

```
sampleTimesFromDist(n, cts)
```

Arguments

<code>n</code>	Integer. Number of time points to sample.
<code>cts</code>	Numeric vector. Pilot TOD values (hours, any range, wrapped to $[0, 24)$ internally).

Value

Numeric vector of length n with sampled TOD values in $[0, 24)$.

```
scp_load_pilot
```

Load a Bundled SCP Pilot Dataset

Description

Loads one standardized pilot object from the bundled pilot database. Each pilot is a pre-fit `CircadianBioOptions` S3 object (fit via the same cosinor pipeline as [estCircadianParam](#)) and can be passed directly to `runSimsSingleCohort()` or `runDifferentialPower()`.

Usage

```
scp_load_pilot(
  species,
  dataset,
  tissue,
  condition = NULL,
  alpha_pilot = 0.01,
  adjust = c("none", "BH"),
  K = 1,
  paired_sigma = TRUE
)
```

Arguments

species	Character, length 1. One of "human", "baboon", "other_primate", "mouse", "rat".
dataset	Character, length 1. Manifest dataset field (typically GEO accession or short label, e.g. "GTEx", "GSE160521").
tissue	Character, length 1. Tissue or region label as recorded in the manifest (e.g. "Liver", "NAc").
condition	Character, length 1, optional. Condition arm (e.g. "Control", "All"). If NULL and the (species, dataset, tissue) triple identifies more than one row, the function errors and prints the available conditions.
alpha_pilot	Numeric in $(0, \text{cap}]$. Rhythmicity threshold used to re-derive <code>prop_rhythmic</code> and the amplitude/phase/sigma distributions from the pilot's stored per-gene rhythm table. Default 0.01 matches the build-time threshold. Only takes effect for pilots rebuilt with a <code>rhythm_fit</code> table; older pilots ignore it (with a warning) and return their frozen build-time distributions.
adjust	Multiple-testing adjustment applied to the per-gene p-values before thresholding at <code>alpha_pilot</code> : "none" (raw cosinor p, default) or "BH" (Benjamini-Hochberg q-value).
K	Harmonic order of the requested pilot: 1 (single-harmonic cosinor, default) or 2 (two-harmonic). For $K = 2$ the two-harmonic variant of the pilot (<code><file>_2H.rds</code>) is loaded; an error is raised if no such variant has been built for the pilot.
paired_sigma	Logical (default TRUE). Keep the per-gene (A , σ) pairing so the simulator draws amplitude and noise from the same pilot gene, preserving the realistic narrow $\tilde{r} = A/\sigma$ distribution (the marginal-power / Fig 1-2 Panel A convention). FALSE decouples sigma from amplitude, giving the wide $\tilde{r}$ used by the effect-size-stratified panels (Fig 1-2 Panels B/C).

Value

A `CircadianBioOptions` object with empirical distributions of mesor, log-sigma, amplitude, phase, and proportion rhythmic, ready for downstream simulation.

Examples

```
## Not run:
bio <- scp_load_pilot("human", "GTEx", "Liver")
bio <- scp_load_pilot("human", "GTEx", "Liver", alpha_pilot = 0.05)
bio <- scp_load_pilot("human", "GTEx", "Liver", alpha_pilot = 0.1, adjust = "BH")
bio <- scp_load_pilot("human", "GSE160521", "NAc", "Control")

## End(Not run)
```

 scp_pilots

List Bundled SCP Pilot Datasets

Description

Reads the bundled pilot manifest (`inst/extdata/pilots/manifest.csv`) and returns it as a `data.frame`. Optionally filters by species and ingestion status so a user can quickly enumerate the pilots they actually have available.

Usage

```
scp_pilots(species = NULL, status = "ingested")
```

Arguments

species	Optional character. One or more of "human", "baboon", "other_primate", "mouse", "rat". NULL (default) returns all species.
status	Character. Filter on the status column. Defaults to "ingested" (datasets actually present on disk). Use NULL or "all" to return every row, including "pending", "verify_failed", "controlled", and "ingest_failed" entries.

Value

A data.frame with one row per (dataset, tissue, condition) pilot. Columns: species, dataset, tissue, condition, n, ngenes, design, assay, sample_type, accession, citation, license, file, status.

Examples

```
## Not run:
  scp_pilots()
  scp_pilots(species = "human")
  scp_pilots(status = "all")

## End(Not run)
```

scp_pilot_database *SCP Pilot Dataset Database*

Description

The SCP package bundles a curated database of public circadian transcriptomic pilot datasets that users can plug into `runSimsSingleCohort()` or `runDifferentialPower()` as the parameter source for power simulation. Each bundled pilot is a pre-fit `CircadianBioOptions` object stored under `inst/extdata/pilots/<species>/`, alongside a CSV manifest that documents provenance (accession, citation, license), study design (active vs passive), and condition arm.

Three user-facing functions are exposed:

`scp_pilots` Return the manifest as a data.frame, optionally filtered by species and ingestion status.

`scp_load_pilot` Load and return one standardized pilot object given species, dataset, tissue, and condition.

`scp_pilot_search` Convenience search: filter the manifest by free-text query, species, tissue regex, and in-vivo vs cell-line sample type.

 scp_pilot_search *Search the SCP Pilot Database*

Description

Convenience filter over the pilot manifest. Combines free-text substring matching against dataset, tissue, citation, and accession fields with optional species and tissue-regex filters and an in-vivo vs cell-line restriction. Useful when a user wants to pick a pilot whose design (tissue, sample size, signal strength) matches their planned study.

Usage

```
scp_pilot_search(
  query = NULL,
  species = NULL,
  tissue_pattern = NULL,
  sample_type = "in_vivo"
)
```

Arguments

query	Character, optional. Case-insensitive substring matched against dataset, tissue, accession, citation. NULL matches everything.
species	Character, optional. Restrict to these species.
tissue_pattern	Character, optional. Regular expression matched against tissue (case-insensitive).
sample_type	Character. Default "in_vivo". Use NULL to include cell-line pilots as well.

Value

A data.frame: rows of the manifest passing every active filter.

Examples

```
## Not run:
  scp_pilot_search("liver")
  scp_pilot_search(species = "mouse", tissue_pattern = "brain|cortex")
  scp_pilot_search(sample_type = NULL)    # include cell lines

## End(Not run)
```

 scp_pilot_tod *Get the time-of-day (TOD) vector for a bundled SCP pilot*

Description

Returns the per-sample sampling times for a pilot, in hours mod 24. Prefers the raw p\$times field when the pilot rds preserved it; otherwise parses the manifest tod_phases column and returns the unique sampling phases.

Usage

```
scp_pilot_tod(species, dataset, tissue, condition = NULL)
```

Arguments

```
species, dataset, tissue, condition
```

manifest tuple identifying the pilot.

Value

Numeric vector of times in hours (mod 24); NULL if no TOD information is available for this pilot.

Examples

```
## Not run:
tod <- scp_pilot_tod("baboon", "GSE98965", "SCN", "All")
hist(tod, breaks = seq(0, 24, by = 1))

## End(Not run)
```

SeqModelSel

*Sequential Model Selection for Joint Rhythmicity***Description**

Sequential Model Selection for Joint Rhythmicity

Usage

```
SeqModelSel(
  action = c(1, 2),
  pv = c(0.01, 0.02),
  alpha = 0.05,
  method = "Fisher_FS"
)
```

Arguments

action	Vector indicating which group to test first
pv	P-values for the tests
alpha	Significance threshold
method	Selection method: "Fisher_BS", "Fisher_FS", "Sidak_BS", "Sidak_FS", "Nominal_BS", "Nominal_FS", "VDA", "AWFisher"

Value

Joint rhythmicity category

simCircadianDiff *Simulate Two-Group Differential Circadian Data*

Description

Simulate data for comparing circadian rhythms between two groups (e.g., control vs treatment, disease vs healthy).

Usage

```
simCircadianDiff(
  ngenes = 5000,
  n1 = 24,
  n2 = 24,
  lBaselineExpr,
  lBaselineExpr2 = NULL,
  lod,
  lod2 = NULL,
  amplitude,
  amplitude2 = NULL,
  sigma_rhythmic = NULL,
  prop_rhythmic = 0.25,
  prop_DR = 0.1,
  prop_DP = 0.2,
  prop_DM = 0,
  phase_diff = c(-6, 6),
  dp_shift_mode = c("fixed", "uniform"),
  amp_diff = c(0.5, 2),
  mesor_diff = c(0.5, 2),
  period = 24,
  design = c("active", "passive"),
  cts = NULL,
  cts2 = NULL,
  sim.seed = 42,
  harmonics = NULL
)
```

Arguments

ngenes	Number of genes
n1	Sample size for group 1
n2	Sample size for group 2
lBaselineExpr	Log baseline expression. Default: <code>rnorm(ngenes, 5, 2)</code>
lBaselineExpr2	As <code>lBaselineExpr</code> but for group 2. <code>NULL</code> (default) reuses group 1's distribution.
lod	Log over-dispersion (noise). Default: <code>rnorm(ngenes, -1, 0.3)</code> , so $\sigma = \exp(\text{lod}) \sim \text{LogNormal}(\mu=-1, \text{sd}=0.3)$ with median ~ 0.37

lod2	As lod but for group 2. NULL (default) shares group 1 values.
amplitude	Amplitude distribution for rhythmic genes in group 1.
amplitude2	As amplitude but for group 2. NULL (default) reuses group 1's distribution.
sigma_rhythmic	Optional numeric vector of per-gene noise values for rhythmic genes (same length as amplitude); when supplied, amplitude and sigma are drawn jointly to preserve pilot A-sigma correlation.
prop_rhythmic	Overall proportion of rhythmic genes
prop_DR	Proportion with differential rhythmicity
prop_DP	Proportion with differential phase (among rhythmic in both)
prop_DM	Proportion with differential mesor (both groups rhythmic, same amplitude/phase, different mean level).
phase_diff	Range of phase shift (hours) for DP genes
dp_shift_mode	"fixed" (use phase_diff[2]) or "uniform" (sample uniformly within the phase_diff range).
amp_diff	Unused; retained for interface compatibility
mesor_diff	Range of mesor shift for DM genes c(min, max) (additive, log-scale units).
period	Period (default 24)
design	"active" or "passive"
cts	TOD distribution for passive design
cts2	As cts but for group 2. NULL (default) reuses cts.
sim.seed	Random seed
harmonics	Numeric length-2 c(alpha2, alpha3) harmonic coefficients for non-cosinor signal generation (default NULL, equivalent to c(0, 0)).

Value

Named list with the following elements:

expr1	Gene expression matrix for group 1 (ngenes x n1).
expr2	Gene expression matrix for group 2 (ngenes x n2).
times1	Numeric vector of sample times for group 1 (hours).
times2	Numeric vector of sample times for group 2 (hours).
ground_truth	data.frame (ngenes rows) with columns: gene, diff_type (0-5), diff_type_label, mesor1, mesor2, amplitude1, amplitude2, phase1, phase2, sigma, sigma2, phase_diff, amp_ratio, is_rhythmic_g1, is_rhythmic_g2.
effectsize_DR1	Per-gene SNR in group 1: $A_{g,1}/\sigma_{g,1}$.
effectsize_DR2	Per-gene SNR in group 2: $A_{g,2}/\sigma_{g,2}$.
effectsize_phase	Per-gene phase-displacement effect size.
effectsize_amp	Per-gene amplitude-difference effect size.
effectsize_mesor	Per-gene mesor-difference effect size.
simOptions	List of simulation parameters used (ngenes, n1, n2, prop_rhythmic, prop_DR, prop_DP, prop_DM, period, design).

Gene expression model

Each gene g follows the cosinor model: $y_g(t) = M_g + A_g * \cos(\omega * t - \omega * \phi_g) + \epsilon_g$ where $\omega = 2\pi/\text{period}$ and $\epsilon_g \sim N(0, \sigma_g^2)$.

Default parameter distributions

When lBaselineExpr and IOD are not provided, parameters are drawn from:

Noise (sigma): $\log(\sigma_g) \sim N(\mu = -1, \text{sd} = 0.3) \Rightarrow \sigma_g \sim \text{LogNormal}(\mu = -1, \text{sd} = 0.3) \Rightarrow \text{median}(\sigma) = \exp(-1) \sim 0.37, \text{IQR} \sim [0.27, 0.49]$

Amplitude (A): $A_g \sim \max(\text{LogNormal}(\mu = \log(0.4), \text{sd} = 0.5), 0.05) \Rightarrow \text{median}(A) = 0.4, \text{IQR} \sim [0.24, 0.66]$ (Clamped below at 0.05 to prevent near-zero amplitudes)

Signal-to-noise ratio ($r = A/\sigma$): Since both A and σ are log-normal, $r = A/\sigma$ is also approximately log-normal with: $\log(r) \sim N(\log(0.4) - (-1), \sqrt{0.5^2 + 0.3^2}) = N(\log(0.4) + 1, 0.58) \sim N(0.08, 0.58) \Rightarrow \text{median}(r) \sim \exp(0.08) \sim 1.08 \Rightarrow$ Most genes fall in $r = 0.3 - 2.0$ (right-skewed distribution)

This produces a realistic scenario where the bulk of target genes have moderate signal strength ($r = 0.5 - 1.5$), few genes have strong signals ($r > 3$), and marginal power is dominated by the crowded low-to-moderate r strata. See Panel F in the stratified power plots.

Types of differential expression

Type 0 Non-rhythmic in both groups

Type 1 Rhythmic in both, same parameters (control)

Type 2 DR: Rhythmic in Group 1 only

Type 3 DR: Rhythmic in Group 2 only

Type 4 DP: Differential phase (both rhythmic, different peak times)

Type 5 DM: Differential mesor (both rhythmic, same A/ϕ , different mean level)

simCircadianDiffFMM

Simulate Two-Group Differential Circadian Data under FMM Waveform

Description

Extends `simCircadianDiff` by replacing the cosinor generative model with the FMM (Frequency Modulated Mobius) waveform. Gene-type assignment (DR/DP/DM/null) and parameter draws are identical to `simCircadianDiff`; only expression generation uses FMM.

Uses a precomputed unit-amplitude FMM template (one per ω value) and per-gene interpolation, avoiding the per-gene `FMM::generateFMM()` call and giving a ~1000x speed-up over naive implementation.

Beta-invariance: The orientation parameter `beta` shifts the peak location but does not alter amplitude. DCP differential-amplitude power (DR, DP) is therefore invariant to `beta`. Only phase estimation (DP peak-time accuracy) is affected by `beta`.

omega = 1 identity: When $\omega = 1$ the FMM waveform reduces exactly to the cosinor model; this function returns the `simCircadianDiff` result unmodified for efficiency.

Usage

```
simCircadianDiffFMM(..., omega = 1, beta = pi)
```

Arguments

<code>...</code>	All other arguments passed directly to <code>simCircadianDiff</code> .
<code>omega</code>	Numeric in $[0, 1]$. FMM waveform shape: 1 = pure cosinor (default); 0 = flat (arrhythmic); values in $(0, 1)$ give increasingly peaked non-sinusoidal waveforms.
<code>beta</code>	Numeric. FMM orientation parameter (default π gives a symmetric peak at α). Does not affect amplitude-based power (beta-invariance).

Value

Same list structure as `simCircadianDiff`.

References

Rueda C, Rodriguez-Collado A, Peddada SD (2019). "A single notable oscillation in humans and mammals with strong implications in chronobiology." *Sci Rep* 9, 17982. doi:10.1038/s41598019-545691

See Also

`simCircadianDiff`, `simCircadianFMM`, `plotFMMDifferential`

<code>simCircadianFMM</code>	<i>Simulate Single-Cohort Circadian Data Under FMM Waveform</i>
------------------------------	---

Description

Generates a synthetic gene expression matrix for one cohort using the Frequency Modulated Mobius (FMM) model. The waveform shape is controlled by `omega`: `omega = 1` reproduces a pure sinusoid (identical to DCP cosinor truth), `omega = 0` produces a flat signal (no rhythmicity), and intermediate values produce a peaked waveform. Gene-level parameters (mesor, amplitude, phase, noise) are drawn from the empirical distributions in `bio.opts` exactly as in `simCircadianSingleCohort`.

Beta-invariance: The orientation parameter `beta` shifts the peak location on the 24-hour cycle but does not alter the amplitude of the fitted cosinor signal. DCP amplitude-based detection power is therefore invariant to `beta`; only phase estimation is affected.

Usage

```
simCircadianFMM(
  bio.opts,
  cts,
  omega = 1,
  beta = pi,
  alpha_fixed = NULL,
  seed = NULL
)
```

Arguments

<code>bio.opts</code>	A <code>CircadianBioOptions</code> object from <code>estCircadianParam</code> .
<code>cts</code>	Numeric vector of sample collection times in hours $[0, 24)$. The length of this vector determines the number of simulated samples (N).
<code>omega</code>	Numeric scalar in $[0, 1]$. FMM waveform shape parameter. <code>omega = 1</code> gives a pure sinusoid (cosinor truth); <code>omega = 0</code> gives a flat (arrhythmic) signal; values in $(0, 1)$ give increasingly non-sinusoidal peaked waveforms. Defaults to <code>1.0</code> .
<code>beta</code>	Numeric scalar. FMM orientation (skewness) parameter. Default π gives a symmetric peak. Does not affect amplitude-based power (beta-invariance); affects only peak-time (phase) estimation.
<code>alpha_fixed</code>	Optional numeric acrophase (radians) to force on every rhythmic gene, overriding the sampled per-gene phase. <code>NULL</code> (default) leaves phases sampled as usual.
<code>seed</code>	Optional integer random seed for reproducibility.

Value

Named list with:

`expr` Gene expression matrix ($n_{\text{genes}} \times N$).
`is_rhythmic` Logical vector of length n_{genes} ; `TRUE` for rhythmic genes.
`r_values` Per-gene signal-to-noise ratio A_g/σ_g .
`omega` The omega value used.

References

Rueda C, Rodriguez-Collado A, Peddada SD (2019). "A single notable oscillation in humans and mammals with strong implications in chronobiology." *Sci Rep* 9, 17982. doi:10.1038/s41598019-545691

See Also

`simCircadianSingleCohort`, `estCircadianParam`, `plotFMMViolation`

`simCircadianSingleCohort`

Simulate Single-Cohort Circadian Gene Expression Data

Description

Generates a synthetic gene expression matrix for one cohort by drawing per-gene parameters (mesor, amplitude, phase, noise) from the empirical distributions stored in a `CircadianBioOptions` object. Rhythmic genes receive nonzero amplitude and a randomly drawn peak phase; non-rhythmic genes have amplitude fixed at zero.

Usage

```
simCircadianSingleCohort(
  bio.opts,
  cts,
  alpha2 = 0,
  alpha3 = 0,
  omega = 1,
  beta = pi,
  seed = NULL
)
```

Arguments

bio.opts	A CircadianBioOptions object from estCircadianParam.
cts	Numeric vector of sample collection times (hours). Length determines the number of simulated samples (N).
alpha2	Second-harmonic coefficient (default 0). Adds a $A \cdot \alpha_2 \cos(4\pi t/T - 4\pi\phi/T)$ term. Only used when $\omega = 1$ (cosinor path). Ignored when $\omega < 1$.
alpha3	Third-harmonic coefficient (default 0). Same conditions as alpha2.
omega	FMM waveform shape parameter in $(0, 1]$ (default 1.0). $\omega = 1$ uses the traditional cosinor generator (with alpha2/alpha3). $\omega < 1$ uses simCircadianFMM , a non-sinusoidal Mobius waveform. The two paths are mutually exclusive: FMM takes precedence when $\omega < 1$.
beta	FMM orientation parameter (default π). Controls the phase of waveform asymmetry. Only used when $\omega < 1$. β -invariance ensures DCP detection power is unaffected by beta.
seed	Optional integer random seed for reproducibility.

Value

Named list with:

expr Gene expression matrix (ngenes x N).
 is_rhythmic Logical vector of length ngenes; TRUE for rhythmic genes.
 r_values Per-gene signal-to-noise ratio A_g/σ_g .

See Also

[estCircadianParam](#), [simCircadianDiff](#)

simCircadianSingleCohort2H

Simulate Single-Cohort Circadian Data Under the Two-Harmonic Cosinor Model

Description

Two-harmonic analog of [simCircadianSingleCohort](#). Generates expression from

$$y_{gi} = \mu_g + A_{g,1} \cos(\omega_0(t_i - \phi_{g,1})) + A_{g,2} \cos(2\omega_0(t_i - \phi_{g,2})) + \epsilon_{gi}$$

using the joint pilot tuple $(A_1, \phi_1, A_2, \phi_2, \sigma)$ stored on `bio.opts` by [estCircadianParam2H](#). The same index is drawn across all five vectors so the pilot's cross-parameter dependence is preserved.

`r_values` is reported on the 1st-harmonic scale $r_g = A_{g,1}/\sigma_g$ to match the existing r-stratification machinery and the manuscript's r_g notation.

Usage

```
simCircadianSingleCohort2H(bio.opts, cts, seed = NULL)
```

Arguments

<code>bio.opts</code>	A <code>CircadianBioOptions</code> produced by estCircadianParam2H (must carry <code>paired_2h = TRUE</code> , <code>amplitude2</code> , <code>phase2</code>).
<code>cts</code>	Numeric vector of sample collection times in hours; length determines N .
<code>seed</code>	Optional integer random seed.

Value

Named list:

`expr` Gene expression matrix ($n_{\text{genes}} \times N$).

`is_rhythmic` Logical vector.

`r_values` Per-gene 1st-harmonic SNR $A_{g,1}/\sigma_g$.

`A1, phi1, A2, phi2` Per-gene truth vectors (length n_{genes} ; zero for non-rhythmic genes).

See Also

[estCircadianParam2H](#), [simCircadianSingleCohort](#)

StopRule

Stop Rule for Sequential Testing

Description

Stop Rule for Sequential Testing

Usage

```
StopRule(action, pv, alpha, method)
```

Arguments

<code>action</code>	Vector indicating which group to test first
<code>pv</code>	P-values
<code>alpha</code>	Significance threshold
<code>method</code>	"FS" (forward stop) or "BS" (basic stop)

Value

List with type and one

stratified_power	<i>Stratified Power Analysis</i>
------------------	----------------------------------

Description

Calculate power stratified by effect size ($r = A/\sigma$) stratified power by expression level.

Usage

```
stratified_power(
  pvals,
  ground_truth,
  effect_sizes,
  breaks = c(0, 0.5, 1, 2, Inf),
  alpha = 0.05
)
```

Arguments

pvals	Vector of p-values
ground_truth	Logical vector of true positives
effect_sizes	Vector of effect sizes ($r = A/\sigma$)
breaks	Break points for effect size strata
alpha	Significance threshold

Value

Data frame with power by effect size stratum

summarizeDCPPairs	<i>Summarize DCP pair results into a wide gene-level table</i>
-------------------	--

Description

Loads saved DCP_Analyze RDS files and computes a wide-format table with: - Sample and gene counts per tissue - TOD coverage per tissue - Single-tissue rhythmicity at multiple p/FDR thresholds - Joint rhythmicity (TOJR) breakdown - DR/DP/DA/DM counts at multiple thresholds - $r = A/\sigma$ for top-300 rhythmic genes (cross-tissue comparable) - Effect sizes: median $|\delta_{\text{phil}}$ (DP), median $|\delta_{\text{MI}}$ (DM) - Recommendation flags ($\geq \text{min_n_recommend}$ genes per endpoint at FDR 5

Usage

```
summarizedDCPPairs(  
  rds_paths,  
  pair_meta = NULL,  
  out_csv = NULL,  
  min_n_recommend = 50L,  
  p_thresholds = c(0.05, 0.01, 0.001),  
  fdr_thresholds = c(0.2, 0.1, 0.05, 0.01),  
  top_k = 300L,  
  species = "Human",  
  dataset = ""  
)
```

Arguments

- rds_paths Named character vector: names = pair tags, values = RDS paths.
- pair_meta Data frame with columns: tag, tissue1, tissue2.
- out_csv Path for output CSV (NULL = return only).
- min_n_recommend Minimum gene count to flag endpoint as sufficient (default 50).
- p_thresholds Raw p-value thresholds (default c(0.05, 0.01, 0.001)).
- fdr_thresholds FDR thresholds (default c(0.20, 0.10, 0.05, 0.01)).
- top_k Number of top rhythmic genes (by p-value) for r statistics (default 300).
- species Character string for species column.
- dataset Character string for dataset column.

Value

Data frame, one row per pair, sorted by sufficient_all3 then score_all3.

SummarizeDR	<i>Summary of Differential Rhythmicity Tests</i>
-------------	--

Description

Summary of Differential Rhythmicity Tests

Usage

```
SummarizeDR(result, test = "DRF", type = "p-value", val = 0.05, out = "long")
```

Arguments

- result List of p-value vectors
- test Test names
- type Types of results (e.g. "p-value")
- val Cutoffs
- out "long" or "wide" format

Value

Summary table

summaryBootstrapDesignGrid	<i>Summarize bootstrap design grid results</i>
----------------------------	--

Description

Summarize bootstrap design grid results

Usage

```
summaryBootstrapDesignGrid(  
  result,  
  test_type = "DR",  
  fdr_threshold = NULL,  
  verbose = TRUE  
)
```

Arguments

result	Output from runBootstrapDesignGrid()
test_type	Test type to summarize ("DR", "DP", "DM")
fdr_threshold	FDR threshold (used for display label only)
verbose	Print table

Value

Invisible data frame with summary

summaryRunPower	<i>Summarize power analysis results in a compact table.</i>
-----------------	---

Description

Computes mean marginal power, type I error, FDR, TD, FD, and FDC across simulation replicates for each sample size.

Usage

```
summaryRunPower(powerOutput, verbose = TRUE)
```

Arguments

powerOutput	Output from runPowerAnalysis()
verbose	Print the table (default TRUE)

Value

Data frame with one row per sample size (invisibly)

targeted_power	<i>Targeted Power (for biologically meaningful effects)</i>
----------------	---

Description

Calculate power only for genes with effect sizes above a meaningful threshold.

Usage

```
targeted_power(
  pvals,
  ground_truth,
  effect_sizes,
  min_effect = 0.5,
  alpha = 0.05
)
```

Arguments

pvals	P-values
ground_truth	Logical vector of true positives
effect_sizes	Effect sizes ($r = A/\sigma$)
min_effect	Minimum effect size to be considered "meaningful"
alpha	Significance threshold

Value

Targeted power (proportion of meaningful effects discovered)

testAnyDifferential	<i>Extract p-values from DCP pipeline for power analysis</i>
---------------------	--

Description

Simplified wrapper that returns just p-values for integration with runSimsDiff()

Usage

```
testAnyDifferential(
  y1,
  y2,
  times1,
  times2,
  period = 24,
  test_types = c("DR", "DP")
)
```

Arguments

y1, y2	Gene expression matrices for groups 1 and 2 (genes x samples).
times1	Time points for group 1
times2	Time points for group 2
period	Period (default 24)
test_types	Character vector of endpoints to test: any of "DR" (differential rhythmicity), "DP" (differential phase).

Value

List with p_DR, p_DP

toTOJR	<i>Types of Joint Rhythmicity (TOJR)</i>
--------	--

Description

Categorize genes into four types of joint rhythmicity: arrhy, rhyI (rhythmic in group 1 only), rhyII (rhythmic in group 2 only), both

Usage

```
toTOJR(  
  x,  
  method = "Sidak_FS",  
  amp.cutoff = 0,  
  alpha = 0.05,  
  adjustP = TRUE,  
  p.adjust.method = "BH",  
  parallel.ncores = 1  
)
```

Arguments

x	Output from DCP_Rhythmicity(x1, x2) with rhythm.joint component
method	Algorithm for categorization: "Sidak_FS", "Sidak_BS", "VDA", "AWFisher"
amp.cutoff	Amplitude cutoff for considering genes rhythmic
alpha	Threshold for rhythmicity p-value
adjustP	Use adjusted p-values?
p.adjust.method	P-value adjustment method
parallel.ncores	Number of cores for parallel processing

Value

Vector of TOJR categories

Examples

```
# Re-calculate TOJR with q-value cutoff 0.1
x = DCP_sim_data(ngene=1000, nsample=30, A1=c(1, 3), A2=c(1, 3),
phase1=c(0, pi/4), phase2=c(pi/4, pi/2),
M1=c(4, 6), M2=c(4, 6), sigma1=1, sigma2=1)
rhythm.res = DCP_Rhythmicity(x1 = x[[1]], x2 = x[[2]])
TOJR.new = toTOJR(rhythm.res, alpha = 0.1, adjustP = TRUE)
```

updateBioOptions	<i>Update CircadianBioOptions (like modifyList but preserves class)</i>
------------------	---

Description

Update CircadianBioOptions (like modifyList but preserves class)

Usage

```
updateBioOptions(opts, ...)
```

Arguments

opts	Existing CircadianBioOptions
...	Named arguments to override

Value

Updated CircadianBioOptions

updateSimOptions	<i>Update simulation options (regenerate rhythmic genes)</i>
------------------	--

Description

Update simulation options (regenerate rhythmic genes)

Usage

```
updateSimOptions(sim.opts)
```

Arguments

sim.opts	Simulation options from createSimOptions()
----------	--

Value

Updated simulation options with new seed and regenerated parameters

Note

Deprecated. Not called by any current runner function. Retained for backward compatibility only.

Index

.buildBioFromBoot, 5
.collapseTail, 5
.computeMarginalPower, 6
.load_pilot_data, 7
.prepDiffStratified, 7
.resampleSubjectIndices, 8
.selectTimePoints, 8
.vcut, 9

add_se_bars, 9
adjust.to.2pi, 10
alpha.adjust, 10
amp.cut, 11

bootstrapParams, 11

calculate_CI.M, 12
calculate_CI_A.phase.Taylor, 12
choose.abs.min, 13
CircadianAnalysisOptions, 13
CircadianBioOptions, 14
CircadianBootstrapOptions, 17
CircadianDesignOptions, 17
CircaPower, 18
circaPowerApproxN80, 19
circular_concentration, 20
circular_difference, 20
circular_mean, 21
compare_methods, 22
compareDesignApproaches, 21
CP_OneGroup, 22

DCP_Analyze, 23
DCP_DiffPar, 24
DCP_DiffR2, 25
DCP_Rhythmicity, 26
DCP_sim_data, 27
detect_CircaCompare, 28
detect_cosinor, 29
detect_DCP, 29, 30
detect_DCP_diff, 30
detect_DODR, 31
detect_FMM, 31, 35
detect_JTK, 33

detect_LimoRhyde, 34
detect_MH, 34
detect_RAIN, 35

estCircadianParam, 36, 37, 39, 90, 99, 100
estCircadianParam2H, 37, 101
estCircadianParamFMM, 37, 39
estCircadianParamTwoGroup, 37, 41
estimate_r_from_data, 42

false_discovery_cost, 43
fishers.p, 43
fitCosinorAll, 44
fitSinCurve, 44
format_for_DCP, 46
forwardStop, 46

generate_power_report, 47
generatePilotData, 47
get_phase, 48
get_r_stratum, 48

launchShiny, 49
LR_deltaR2, 50
LR_diff, 51

makeAdaptiveRStrata, 51

npower, 52

one_cosinor_OLS, 53

plot.SCPDiffResult, 54
plot.SCPRecommendResult, 54
plot.SCPSingleResult, 55
plot_power_curves, 65
plot_stratified_power, 65
plot_two_group_heatmap, 66
plotBootstrapDesignGrid, 55
plotBvsMPower, 56
plotDensityResults, 57
plotDesignComparison, 57
plotDiffPower, 58
plotDRPowerStratified, 59

plotFMMDifferential, 60, 62, 98
plotFMMViolation, 60, 61, 99
plotFourierDeviation, 62
plotGroundTruthComparison, 62
plotSingleCohortPower, 63
power_assessment, 66
prepCircadianData, 37, 67
print.CircadianAnalysisOptions,
68
print.CircadianBioOptions, 69
print.CircadianBootstrapOptions,
69
print.CircadianDesignOptions, 70
print.estimate_r_from_data, 70
print.npower, 71
print.SCPDiffResult, 71
print.SCPRecommendResult, 72
print.SCPSingleResult, 72
printDensitySummary, 73
printMethodGuidance, 73

rayleigh_test_circular, 74
recommendDesign, 74, 86
replotDRPower, 75
run_DCP_pipeline, 89
runBootstrapDesignGrid, 76
runDifferentialPower, 78, 84
runFourierDeviationPower, 79
runGroundTruthComparison, 80
runPhaseShiftAnalysis, 81
runPowerAnalysis, 81
runSCPValidation, 82
runSimsDiff, 82
runSimsSingleCohort, 84
runSingleCohortGrid, 86
runSingleCohortPower, 87
runTwoStagePower, 88

sampleTimesFromDist, 90
scp_load_pilot, 90, 92
scp_pilot_database, 92
scp_pilot_search, 92, 93
scp_pilot_tod, 93
scp_pilots, 91, 92
SeqModelSel, 94
simCircadianDiff, 95, 97, 98, 100
simCircadianDiffFMM, 60, 97
simCircadianFMM, 62, 98, 98, 100
simCircadianSingleCohort, 98, 99,
99, 101
simCircadianSingleCohort2H, 39,
100
StopRule, 101

stratified_power, 102
summarizeDCPPairs, 102
SummarizeDR, 103
summaryBootstrapDesignGrid, 104
summaryRunPower, 104

targeted_power, 105
testAnyDifferential, 105
toTOJR, 106

updateBioOptions, 107
updateSimOptions, 107